

Einführung in die C#-Programmierung

Version 1.0.0, 30. Januar 2026

Author: CHA, SLM | C#-Adaption: Kimi K2.5 | Prompter: Georg Graf (GRG)

Inhaltsverzeichnis

1. Grundlagen	1
1.1. Entwicklungsumgebung JetBrains Rider	1
1.2. Alternative Entwicklungsumgebungen	2
2. Namens- und Coding-Konventionen	3
2.1. Allgemeine Grundsätze	3
2.2. Namenskonventionen	3
2.3. Code-Formatierung	3
2.4. Fail-Fast Pattern	4
2.5. Kommentare und Dokumentation	5
2.6. Verzeichnisstruktur und Organisation	5
3. Professionelle Entwicklungswerkzeuge	6
3.1. Professionelle IDEs: Rider und Visual Studio	6
3.2. Projektstruktur und Solutions	6
3.3. Debugger	6
4. Unit-Tests und TDD (xUnit)	7
4.1. Test-Driven Development (TDD)	7
4.2. Testprojekt einrichten	7
4.3. Facts und Theories	7
4.4. Setup und Lifecycle	8
4.5. Wichtige Assertions	8
4.6. Tests ausführen	8
5. Klassen, Objekte und Methoden	10
5.1. Properties (Eigenschaften)	11
5.2. Auto-Properties	12
5.3. Kommentare	12
5.4. Konstruktoren	12
5.5. Methoden	13
5.6. Signatur von Methoden	14
5.7. Parameter-Prüfungen (Fail-Fast)	15
5.8. Lokale Variablen, Berechnungen und Typkonvertierung	16
6. Variablen, Datentypen und Methoden	17
6.1. Werttypen	17
6.2. Referenztypen	17
6.3. Methoden (Fortsetzung)	18
6.4. Parameterübergabe mit <code>ref</code> und <code>out</code>	19
6.5. Wann ist die Verwendung von <code>ref</code> und <code>out</code> sinnvoll?	19
7. Objekt-Beziehungen	20
8. Strings	23
8.1. String-Vergleich	24
8.2. StringBuilder	24
9. Schleifen	25
9.1. <code>while</code> -Schleife	25
9.2. <code>do-while</code> -Schleife	25
9.3. <code>for</code> -Schleife	25
9.4. <code>foreach</code> -Schleife	25

9.5. Codebeispiele und Code-Wiederverwendung	25
10. Arrays und Collections	29
10.1. Arrays	29
10.2. Generische Listen (List<T>)	29
10.3. Weitere Collection-Typen	29
10.4. Manuelle Array-Verwaltung	30
11. Statische Methoden und Nützliche Klassen	32
11.1. Statische Methoden	32
11.2. Die <code>Main</code> -Methode	32
11.3. Nützliche Klassen: Math und Random	32
12. Vererbung	34
12.1. Grundlagen der Vererbung	34
12.2. Der Zugriffsmodifizierer <code>protected</code>	35
12.3. Methoden überschreiben vs. Überladen	36
12.4. Polymorphie in der Praxis	37
12.5. Abstrakte Klassen und Methoden	38
12.6. Vererbung vs. Komposition	39
12.7. Die <code>sealed</code> -Klasse und <code>sealed</code> -Methoden	41
12.8. Zusammenfassung Vererbung	41
13. Interfaces	43
13.1. Wann braucht man Interfaces?	43
13.2. Praktisches Beispiel: <code>IUsable</code> mit Waffen und Heiltränken	43
13.3. Die List mit Interface-Typisierung	45
13.4. Interfaces vs. Abstrakte Klassen	46
13.5. Funktionale Interfaces und Delegates	48
13.6. Repository-Pattern mit Interfaces	49
13.7. Zusammenfassung	51
14. Datenbanken mit Entity Framework Core (EF Core)	52
14.1. EF Core Pakete installieren	52
14.2. Das Domänenmodell und der DbContext	52
14.3. Schema-Migrations	52
14.4. CRUD-Operationen	53
14.5. 1:n Beziehungen (One-to-Many)	53
14.6. n:m Beziehungen (Many-to-Many)	54
14.7. Laden von Beziehungen (Eager Loading)	54
14.8. Best Practices für EF Core	55
15. LINQ (Language Integrated Query)	56
15.1. Grundlegende LINQ-Operationen	56
15.2. LINQ with Objects	56
15.3. Query-Syntax vs. Method-Syntax	57
16. Exception Handling	58
16.1. Grundlagen	58
16.2. Eigene Exceptions	58
17. Razor	59
17.1. Razor-Grundlagen	59
17.2. Razor-Page Model	59

17.3. Razor-Komponenten	60
18. Razor mit HTMX	61
18.1. Was ist HTMX?	61
18.2. HTMX in Razor Pages	61
18.3. PageModel für HTMX	62
18.4. HTMX-Partial Views	63
18.5. HTMX-Trigger und Events	63
18.6. HTMX mit Animationen	64
18.7. Vorteile von Razor + HTMX	65
18.8. Best Practices für Razor + HTMX	65

Kapitel 1. Grundlagen

Dieses Skriptum beinhaltet die Grundlagen der Programmierung in der Programmiersprache C#. Programmieren durchdringt unser Leben heute in vielen Bereichen, die resultierende Software findet sich in zahlreichen Bereichen: mobile Applikationen auf Smartphones und Tablets, Webanwendungen, Spiele, Server-Anwendungen, Cloud-Computing u.v.m. Die hier vorgestellten Konzepte stellen die Grundlage für all jene Anwendungsgebiete dar, der primäre Fokus liegt auf der Entwicklung von *Backends*, also Programmen die auf Servern laufen und die *Frontends* (also die Clients) mit den entsprechenden Daten und Berechnungen versorgen.

Programmierung bedeutet grundsätzlich, eine Folge von Operationen auf einem Mikroprozessor (CPU) anzusteuern, welche die Daten im Hauptspeicher oder Permanentspeicher verarbeiten. Die Programmierung auf Prozessebene (Assembler-Sprachen) ist jedoch sehr aufwändig, und wird heutzutage nur noch für wenige Spezialanwendungen genutzt. Die sogenannten höheren Programmiersprachen wie beispielsweise Java, C# oder Python setzen auf einer höheren Abstraktionsebene an, die mehr dem menschlichen Denken entspricht, und eine schnellere und robustere Programmentwicklung ermöglicht.

Das Programm besteht dabei zunächst aus dem Quelltext oder Code, d.h. ein oder mehrere Dateien im reinen Text-Format. Um das Programm tatsächlich ausführen zu können, muss es zunächst *übersetzt* werden. Diese Aufgabe übernimmt der *Compiler*. Enthält der Programmcode Fehler, konkret Elemente die nicht der Vorgabe der Programmiersprache entsprechen, so werden entsprechende Fehlermeldungen erzeugt, die dem Programmierer oder der Programmiererin bei der Behebung ebendieser helfen. Konnte das Programm zur Gänze übersetzt (compiliert) werden, so kann es anschließend ausgeführt werden. Die reine compiler-fähigkeit des Programmes sagt natürlich noch nichts über seine Korrektheit aus. Viele Fehler machen sich erst zur Laufzeit bemerkbar, und müssen anschließend behoben werden. In der modernen Programmierung versucht man die Anzahl der Fehler in Programmen durch *Tests* zu minimieren. Das Aufspüren von Fehlern oder unerwartetem Verhalten in Programmen wird durch *Debugger* unterstützt, welche es ermöglichen das Programm an bestimmten Stellen anzuhalten und aktuelle Werte zu analysieren.

Bei C# wird der Programmcode vom Compiler in eine Zwischensprache (IL - Intermediate Language) übersetzt. Diese wird dann auf der Common Language Runtime (CLR), einer virtuellen Maschine, ausgeführt. Dieses Konzept ermöglicht es die resultierenden Programme auf verschiedenen Betriebssystemen und Hardware-Architekturen auszuführen (dank .NET Core/.NET 5+).

Die Werkzeuge Compiler, Debugger, Text-Editor und vieles mehr werden üblicherweise in einer sogenannten *Entwicklungsumgebung* (IDE - Integrated Development Environment) verwendet. Diese ermöglicht einen komfortablen und intuitiven Umgang mit allen benötigten Werkzeugen. Wir verwenden im Unterricht primär **JetBrains Rider**.

1.1. Entwicklungsumgebung JetBrains Rider

JetBrains Rider ist eine plattformunabhängige IDE, die auf der IntelliJ-Plattform und ReSharper basiert. Sie bietet eine exzellente Unterstützung für .NET-Entwicklung auf Windows, macOS und Linux.

Key Points: * **Intelligente Code-Analyse:** Erkennt Fehler bereits während des Tippens und schlägt Optimierungen vor (basierend auf der ReSharper-Engine). * **Refactoring:** Mächtige Werkzeuge zum sicheren Umbenennen oder Verschieben von Code-Teilen. * **Integrierte Werkzeuge:** Debugger, Unit-Test-Runner und Git-Unterstützung sind nahtlos integriert. * **Performance:** Schnelle Indizierung und flüssiges Arbeiten auch in großen Projekten.

1.2. Alternative Entwicklungsumgebungen

Je nach Anwendungsfall oder persönlicher Präferenz können auch andere IDEs verwendet werden:

1.2.1. Microsoft Visual Studio

Der Industriestandard für die Windows-Entwicklung. * **Key Points:** Umfassendste Unterstützung für Windows-spezifische Technologien (WPF, WinForms), sehr mächtiger Debugger, jedoch nur für Windows verfügbar.

1.2.2. Visual Studio Code (VS Code)

Ein leichtgewichtiger, plattformunabhängiger Editor, der durch Plugins zur vollwertigen Entwicklungsumgebung ausgebaut werden kann. * **Key Points:** Extrem schnell, sehr flexibel und kostenlos. * **Empfohlene Plugins für C#:** C# Dev Kit: **Die offizielle Erweiterung von Microsoft für Solution-Management und Testing.** C#: Bietet Basis-Sprachunterstützung, IntelliSense und Debugging. .NET Install Tool:** Hilft bei der Verwaltung der benötigten .NET SDKs.

1.2.3. Projekt erstellen mittels .NET CLI

Unabhängig von der gewählten IDE können Projekte über die Kommandozeile (Terminal) erstellt werden:

```
# Erstellen eines neuen Konsolenprojekts
dotnet new console -n MeinProjekt
cd MeinProjekt

# Projekt öffnen
code .
```

In VS Code öffnet sich das Projekt. Die Datei `Program.cs` enthält den Einstiegspunkt des Programms. Sie können das Programm mit `F5` oder über das Menü "Run" starten.

Kapitel 2. Namens- und Coding-Konventionen

Bevor wir mit der eigentlichen Programmierung beginnen, ist es wichtig, sich mit den gängigen Konventionen vertraut zu machen. Diese Konventionen werden zwar vom Compiler nicht erzwungen, sind aber unbedingt einzuhalten, um lesbaren, wartbaren und professionellen Code zu schreiben.

2.1. Allgemeine Grundsätze

- **Konsistenz:** Verwenden Sie einen Stil konsequent durchgehend im gesamten Projekt
- **Klarheit:** Code sollte selbsterklärend sein - gute Namen sind wichtiger als Kommentare
- **Einfachheit:** Bevorzugen Sie einfache, direkte Lösungen gegenüber cleveren, komplizierten
- **Fail-Fast:** Validieren Sie Eingaben so früh wie möglich und werfen Sie Exceptions bei ungültigen Werten

2.2. Namenskonventionen

Element	Konvention	Beispiel
Klassen	PascalCase	Student, MitarbeiterRepository
Interfaces	PascalCase, beginnt mit I	IRepository, IStudentService
Methoden	PascalCase	GetName(), BerechneAlter()
Eigenschaften (Properties)	PascalCase	Name, Geburtsjahr
Felder (private)	camelCase mit Unterstrich	_name, _geburtsjahr
Konstanten	PascalCase oder UPPER_SNAKE_CASE	MaxGroesse oder MAX_GROESSE
Parameter	camelCase	neuerName, geburtsjahr
Lokale Variablen	camelCase	ergebnis, zaehler
Enum-Typen	PascalCase	Status, Farbe
Enum-Werte	PascalCase	Aktiv, Inaktiv

2.3. Code-Formatierung

```
// Beispiel für korrekte Formatierung
public class Student
{
    private string _name;
    private int _geburtsjahr;

    public Student(string name, int geburtsjahr)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));
        if (geburtsjahr < 1900 || geburtsjahr > DateTime.Now.Year)
            throw new ArgumentOutOfRangeException(nameof(geburtsjahr));

        _name = name;
        _geburtsjahr = geburtsjahr;
    }

    public string Name
```

```
{
    get => _name;
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Name darf nicht leer sein", nameof(value));
        _name = value;
    }
}

public int Geburtsjahr => _geburtsjahr;

public int BerechneAlter(int aktuellesJahr)
{
    if (aktuellesJahr < _geburtsjahr)
        throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

    return aktuellesJahr - _geburtsjahr;
}
}
```

Wichtige Formatierungsregeln:

- **Einrückung:** 4 Leerzeichen pro Ebene (keine Tabs)
- **Klammern:** Öffnende Klammer in neuer Zeile (Allman-Style)
- **Leerzeichen:** Nach Schlüsselwörtern (`if`, `for`, `while`), vor und nach Operatoren
- **Zeilenlänge:** Maximal 120 Zeichen pro Zeile
- **Leerzeilen:** Trennen Sie logische Code-Blöcke mit Leerzeilen

2.4. Fail-Fast Pattern

Das Fail-Fast-Pattern besagt, dass Fehler so früh wie möglich erkannt und behandelt werden sollen. Das bedeutet:

1. **Parameter-Validierung:** Prüfen Sie Parameter sofort am Beginn einer Methode
2. **Exceptions werfen:** Bei ungültigen Werten sofort eine Exception werfen, nicht nur eine Fehlermeldung ausgeben
3. **Keine Stille Fehler:** Vermeiden Sie, dass ungültige Werte stillschweigend korrigiert werden
4. **Guard Clauses:** Verwenden Sie frühe Rückgaben oder Exceptions um verschachtelte If-Blöcke zu vermeiden

Vergleich Happy-Path vs. Fail-Fast:

```
// Happy-Path (NICHT empfohlen) - als Property
public int Groesse
{
    get => _groesse;
    set
    {
        if (value >= 110 && value <= 220)
        {
            _groesse = value;
        }
    }
}
```



```

        else
        {
            Console.WriteLine("Fehler: Größe ungültig");
            _groesse = 170; // Stille Korrektur
        }
    }
}

// Fail-Fast (empfohlen) - als Property mit Guard Clause
public int Groesse
{
    get => _groesse;
    set
    {
        if (value < 110 || value > 220)
            throw new ArgumentOutOfRangeException(nameof(value),
                "Größe muss zwischen 110 und 220 cm liegen");

        _groesse = value;
    }
}

```

2.5. Kommentare und Dokumentation

Verwenden Sie XML-Dokumentation für öffentliche APIs:

```

/// <summary>
/// Berechnet das Alter einer Person basierend auf dem aktuellen Jahr.
/// </summary>
/// <param name="aktuellesJahr">Das aktuelle Jahr</param>
/// <returns>Das Alter in Jahren</returns>
/// <exception cref="ArgumentException">Wenn aktuellesJahr vor dem Geburtsjahr liegt</exception>
public int BerechneAlter(int aktuellesJahr)
{
    if (aktuellesJahr < _geburtsjahr)
        throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

    return aktuellesJahr - _geburtsjahr;
}

```

2.6. Verzeichnisstruktur und Organisation

Organisieren Sie Ihr Projekt nach dem folgenden Muster:

```

MeinProjekt/
├── src/
│   └── MeinProjekt/
│       ├── Models/           # Domänenmodelle
│       ├── Services/         # Geschäftslogik
│       ├── Repositories/     # Datenzugriff
│       ├── Exceptions/       # Eigene Exception-Typen
│       ├── Utils/            # Hilfsfunktionen
│       └── Program.cs         # Einstiegspunkt
├── tests/
│   └── MeinProjekt.Tests/
└── MeinProjekt.sln

```

Kapitel 3. Professionelle Entwicklungswerkzeuge

3.1. Professionelle IDEs: Rider und Visual Studio

Für die professionelle C#-Entwicklung sind **JetBrains Rider** und **Microsoft Visual Studio** die führenden Werkzeuge. Beide bieten einen enormen Funktionsumfang, der über einfache Texteditoren weit hinausgeht.

Gemeinsame Features:

- * **IntelliSense / Code Completion:** Intelligente Code-Vervollständigung mit Kontextinformationen.
- * **Refactoring:** Automatisches Umbenennen, Extrahieren von Methoden oder Klassen, sowie sicheres Verschieben von Code-Teilen.
- * **Fortgeschrittener Debugger:** Haltepunkte mit Bedingungen, Überwachung von Variablen (Watch) und Analyse des Call-Stacks.
- * **NuGet-Integration:** Einfache Verwaltung von externen Bibliotheken über den Paketmanager.
- * **Test Runner:** Komfortable Oberflächen zum Ausführen und Verwalten von Unit-Tests (xUnit, NUnit).

3.2. Projektstruktur und Solutions

In der .NET-Welt werden Projekte üblicherweise in **Solutions** (`.sln`) organisiert. Eine Solution kann mehrere Projekte enthalten (z.B. das Hauptprogramm und ein separates Testprojekt).

Ein typisches .NET-Projekt hat folgende physische Struktur:

```

MeineLoesung/
├── MeineLoesung.sln          # Solution-Datei
├── MeinProjekt/             # Projektverzeichnis
│   ├── MeinProjekt.csproj    # Projektdatei
│   ├── Program.cs           # Einstiegspunkt
│   └── ...                   # Weitere Quellcodedateien
└── MeinProjekt.Tests/       # Testprojekt
    └── ...

```

3.3. Debugger

Der Debugger ermöglicht es, die Ausführung des Programmes anzuhalten, um den aktuellen Zustand (also die aktuellen Werte der Variablen und Eigenschaften) zu überprüfen. Diese Methode ermöglicht Fehler relativ einfach zu lokalisieren und zu beheben.

In modernen IDEs wie **JetBrains Rider**, **Visual Studio** oder **VS Code** setzt man dazu am linken Rand neben der Zeilennummer im Code-Editor einen sogenannten *Breakpoint* (Haltepunkt). Die nächste Ausführung des Programmes stoppt dann bei genau dieser Codezeile. Das Debugger-Fenster zeigt dann die Werte der Felder sowie die lokalen Variablen an.

Mittels "Step Over" (F10) wird das Programm bis nach der nächsten Codezeile ausgeführt. "Step Into" (F11) arbeitet auch etwaige Methodenaufrufe nur Zeile für Zeile ab. "Continue" (F5) setzt hingegen die normale Ausführung des Programmes fort.

Kapitel 4. Unit-Tests und TDD (xUnit)

Um Fehler in der zu entwickelnden Software zu vermeiden, ist die Durchführung von zahlreichen Tests unerlässlich. In .NET verwenden wir häufig das **xUnit**-Framework für Unit-Tests.

4.1. Test-Driven Development (TDD)

TDD ist eine Methodik, bei der Tests **vor** dem eigentlichen Code geschrieben werden. Dies folgt dem **Red-Green-Refactor** Zyklus: 1. **Red**: Schreiben Sie einen Test für eine neue Funktion. Da die Funktion noch nicht existiert, schlägt der Test fehl. 2. **Green**: Schreiben Sie gerade so viel Code, dass der Test besteht. 3. **Refactor**: Verbessern Sie den Code (Lesbarkeit, Performance), während Sie sicherstellen, dass alle Tests weiterhin bestehen.

4.2. Testprojekt einrichten

Ein Testprojekt wird separat zur eigentlichen Anwendung erstellt:

```
# Erstellen eines xUnit Testprojekts
dotnet new xunit -n MeinProjekt.Tests
cd MeinProjekt.Tests

# Referenz auf das zu testende Projekt hinzufügen
dotnet add reference ../MeinProjekt/MeinProjekt.csproj
```

4.3. Facts und Theories

In xUnit gibt es zwei Hauptarten von Tests:

1. **[Fact]**: Ein Test, der immer wahr sein muss. Er hat keine Parameter.
2. **[Theory]**: Ein Test, der für verschiedene Datensätze geprüft wird. Mittels `[InlineData]` können Parameter übergeben werden.

```
using Xunit;

public class PersonTests
{
    // Fact: Ein einzelner Testfall
    [Fact]
    public void Konstruktor_MitGueltigenWerten_ErzeugtPerson()
    {
        // Arrange
        var person = new Person("Max Maier", true, 180, 80);

        // Act & Assert
        Assert.Equal("Max Maier", person.Name);
    }

    // Theory: Testen mit mehreren Datensätzen
    [Theory]
    [InlineData(180, 80, 24.69)]
    [InlineData(170, 50, 17.30)]
    [InlineData(190, 100, 27.70)]
    public void BerechneBmi_BerechnetKorrekt(int groesse, int gewicht, double erwarteterBmi)
    {
        // Arrange
```

```

var person = new Person("Test", true, groesse, gewicht);

// Act
double bmi = person.BerechneBmi();

// Assert
Assert.Equal(erwarteterBmi, bmi, 2); // 2 Nachkommastellen Toleranz
}

[Fact]
public void Konstruktor_MitUnguelteigerGroesse_WirftException()
{
    // Act & Assert
    Assert.Throws<ArgumentOutOfRangeException>(() =>
        new Person("Max Maier", true, 50, 80));
}
}

```

4.4. Setup und Lifecycle

In xUnit wird für **jeden einzelnen Test** eine neue Instanz der Testklasse erzeugt. * **Setup**: Gemeinsamer Vorbereitungscode (z.B. Erzeugen von Testdaten) wird einfach in den **Konstruktor** der Testklasse geschrieben. * **Cleanup**: Falls Ressourcen freigegeben werden müssen (z.B. Dateien oder Datenbankverbindungen), implementiert die Testklasse das Interface `IDisposable` und nutzt die `Dispose()`-Methode.

4.5. Wichtige Assertions

Die Klasse `Assert` bietet zahlreiche Methoden zur Prüfung von Ergebnissen:

Methode	Bedeutung
<code>Assert.Equal(exp, act)</code>	Prüft auf Gleichheit (Werte oder Strings).
<code>Assert.NotEqual(exp, act)</code>	Prüft auf Ungleichheit.
<code>Assert.Null(obj)</code> / <code>Assert.NotNull(obj)</code>	Prüft, ob eine Referenz null (oder nicht null) ist.
<code>Assert.True(cond)</code> / <code>Assert.False(cond)</code>	Prüft eine logische Bedingung.
<code>Assert.Contains(sub, str)</code>	Prüft, ob ein Teilstring oder ein Element in einer Liste vorhanden ist.
<code>Assert.Throws<T>(...)</code>	Prüft, ob der Code eine Exception vom Typ T wirft.
<code>Assert.InRange(val, low, high)</code>	Prüft, ob ein Wert innerhalb eines Bereichs liegt.

4.6. Tests ausführen

Tests können auf verschiedene Arten gestartet werden:

1. **Kommandozeile**: Geben Sie `dotnet test` im Projektverzeichnis ein.
2. **JetBrains Rider**: Nutzen Sie das "Unit Tests" Fenster (Strg+Alt+U) oder klicken Sie auf die Symbole neben den Testmethoden im Editor.
3. **VS Code**: Über den "Testing" View in der Seitenleiste (Symbol mit dem Reagenzglas) oder via `C# Dev Kit`.

Ziel ist es, eine möglichst hohe Testabdeckung des Codes zu erreichen. Das Ausführen dieser Tests liefert dann quasi per Knopfdruck als Ergebnis, ob (nach etwaigen Änderungen) Fehler im Code vorhanden sind.

Kapitel 5. Klassen, Objekte und Methoden

Klassen sind ein grundlegendes Konzept moderner Programmiersprachen. Sie enthalten den wesentlichen Programmcode, ganze Programme entstehen durch ein Zusammenspiel mehrerer Klassen.

Klassen können unterschiedlichste Aufgaben in Programmen übernehmen. Wir wollen aber zunächst eines der wichtigsten Einsatzgebiete betrachten. Häufig modellieren Klassen Entitäten aus der realen Welt. Eine Anwendung könnte beispielsweise dazu dienen *Studenten* einer Schule zu verwalten. Dazu erstellen wir zunächst eine Klasse die einen Studenten modelliert.

```
public class Student
{
    private string _name;
    private int _geburtsjahr;
}
```

Dieser Student hat nur zwei **Eigenschaften** (auch **Attribute** oder **Felder** genannt), nämlich einen Namen und ein Geburtsjahr. Der Name (`_name`) ist dabei eine Text-Zeichenkette (`string`), das Geburtsjahr (`_geburtsjahr`) ist eine ganze Zahl (`int`). Die Angabe `private` legt fest, dass diese Elemente außerhalb der Klasse nicht sichtbar sind.

Es ist unbedingt zu beachten, dass Groß- und Klein-Schreibung eine Rolle spielen. Der Name der Klasse (in diesem Fall `Student`) beginnt stets mit einem Großbuchstaben (PascalCase). Die Felder (`_name` und `_geburtsjahr`) beginnen stets mit einem Unterstrich und Kleinbuchstaben (camelCase mit Unterstrich-Präfix).

Die Strichpunkte am Ende einer Zeile sind hier ebenso wesentlich. Sie beenden eine sogenannte *Anweisung*. Werden sie weggelassen, kann das Programm nicht übersetzt werden.

Beachten Sie weiters die Formatierung dieses Codestückes. Die beiden Zeilen zu den Feldern sind eingerückt (4 Leerzeichen), die schließende geschwungene Klammer ist wieder ganz links gesetzt. Dies ist eine wichtige Konvention, die unbedingt einzuhalten ist.

Wenden wir uns nun der Bedeutung von Klassen zu. Eine Klasse modelliert eine Entität aus der realen Welt, wie zum Beispiel Studenten, Personen, Aktien, Wohnungen, Fahrzeuge, usw. Konkret wird festgelegt, welche Daten diese Entitäten enthalten sollen, in unserem Beispiel einen Namen und ein Geburtsjahr. Die Klasse selbst dient im Programm als *Vorlage* für konkrete Daten. Konkrete Daten zur Klasse `Student` mit `Name` und `Geburtsjahr` können beispielsweise sein: Peter (1981), Sandra (1985), Ercan (1991), Sarah (1994); derartige konkrete Ausprägungen einer Klasse werden **Objekte** genannt. Erstellt man also einen konkreten Studenten, so erzeugt man im Programm ein **Objekt** oder eine **Instanz** einer Klasse. Die Klasse stellt also einen "Bauplan" dar, während Objekte oder Instanzen dann konkrete Ausprägungen desselben sind.

Wir werden in weiterer Folge kennenlernen wie auf die Eigenschaften zugegriffen werden kann (Abfragen von Informationen eines Objektes), und wie diese verändert werden können. Weiters werden wir die Erzeugung von Objekten näher betrachten. Am Ende des Abschnittes werden Sie eine wichtige weitere Eigenschaft von Klassen kennenlernen. Sie können nämlich

nicht nur Daten enthalten, sondern auch *Methoden*. Dies sind Programmstücke, die auf den Daten Berechnungen vornehmen, oder auch Veränderungen vornehmen. Klassen organisieren also das Zusammenspiel von Daten und Programmlogik, Programme selbst entstehen durch die Interaktion von mehreren Objekten.

5.1. Properties (Eigenschaften)

In C# verwenden wir zur Kapselung von Daten nicht direkt Get- und Set-Methoden wie in Java, sondern sogenannte **Properties** (Eigenschaften). Dies ist eine elegantere und kompaktere Syntax.

5.1.1. Getter-Properties (read-only)

Eine read-only Property ermöglicht nur das Lesen eines Wertes:

```
public class Student
{
    private string _name;
    private int _geburtsjahr;

    public string Name => _name;

    public int Geburtsjahr => _geburtsjahr;
}
```

Die Syntax `>` ist eine Expression-bodied Property. Sie ist äquivalent zu:

```
public string Name
{
    get { return _name; }
}
```

5.1.2. Setter-Properties

Properties können auch Schreibzugriff ermöglichen. Wir nutzen dabei das Fail-Fast-Pattern für Validierungen:

```
public class Student
{
    private string _name;
    private int _geburtsjahr;

    public string Name
    {
        get => _name;
        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Name darf nicht leer sein", nameof(value));
            _name = value;
        }
    }

    public int Geburtsjahr
    {
        get => _geburtsjahr;
    }
}
```

```

        set
        {
            if (value < 1900 || value > DateTime.Now.Year)
                throw new ArgumentOutOfRangeException(nameof(value),
                    "Geburtsjahr muss zwischen 1900 und dem aktuellen Jahr liegen");
            _geburtsjahr = value;
        }
    }
}

```

Das Schlüsselwort `value` repräsentiert den übergebenen Wert im Setter.

5.2. Auto-Properties

Für einfache Fälle ohne Validierungslogik können Auto-Properties verwendet werden:

```

public class Student
{
    public string Name { get; set; }
    public int Geburtsjahr { get; set; }
}

```

Der Compiler erzeugt automatisch ein privates Feld im Hintergrund. Für öffentliche APIs mit Validierungsanforderungen sollten jedoch explizite Properties bevorzugt werden.

5.3. Kommentare

Fallweise möchte man im Programm ergänzende Kommentare aufnehmen. Diese stehen zwar im Quelltext, werden aber vom Compiler ignoriert.

```

/*
Dies ist ein mehrzeiliges Kommentar. Am Beginn einer Klasse kann es zum Beispiel
Informationen zum Zweck der Klasse, Autor, Erstellungsdatum etc. enthalten.
*/
public class Student
{
    // dies ist ein einzeiliger Kommentar

    private string _name;
    private int _geburtsjahr;

    public string Name => _name; // Kommentare können auch nach Code stehen

    /*
    public int Geburtsjahr => _geburtsjahr;
    */
}

```

5.4. Konstruktoren

Der Konstruktor legt fest, *wie* ein Objekt erzeugt wird, also welche Werte dabei den Feldern zugewiesen werden können. Wir verwenden den Konstruktor, um die Validierungslogik aus den Properties wiederzuverwenden.

```

public class Student
{

```



```
private string _name;
private int _geburtsjahr;

public Student()
{
    _name = "n/a";
    _geburtsjahr = 1970;
}

public Student(string name, int geburtsjahr)
{
    if (string.IsNullOrEmpty(name))
        throw new ArgumentException("Name darf nicht leer sein", nameof(name));
    if (geburtsjahr < 1900 || geburtsjahr > DateTime.Now.Year)
        throw new ArgumentOutOfRangeException(nameof(geburtsjahr));

    _name = name;
    _geburtsjahr = geburtsjahr;
}

// Properties hier ausgelassen
}
```

Konstruktoren können ebenso wie Methoden überladen werden - eine Klasse kann mehrere Konstruktoren mit unterschiedlichen Parameterlisten haben.

5.5. Methoden

Methoden sind Bestandteile von Klassen, die bestimmte Aufgaben durchführen, wie zum Beispiel Änderungen von Werten, Durchführung von Berechnungen, etc. Sie haben schon spezielle Methoden kennengelernt, nämlich Konstruktoren. Allgemein bestehen Methoden aus einem *Methodennamen*, einem *Rückgabewert*, *Parametern*, sowie einem *Methodenrumpf*, der den eigentlichen Code enthält. *Methodennamen*, *Rückgabewert* und *Parameter* (sowie der Zugriffs-Modifizier) bilden den *Methodenkopf*. Wir betrachten folgende Methode zur Berechnung des aktuellen Alters einer Person:

```
public class Student
{
    private string _name;
    private int _geburtsjahr;

    public int BerechneAlter(int aktuellesJahr)
    {
        if (aktuellesJahr < _geburtsjahr)
            throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

        return aktuellesJahr - _geburtsjahr;
    }
}
```

Die Methode `BerechneAlter` bekommt als Parameter das aktuelle Jahr übergeben. Das Alter der Person ergibt sich dann aus der Differenz von `aktuellesJahr` und `_geburtsjahr`. Im Methodenkopf wurde der Typ des Rückgabewertes mit `int` festgelegt. Die Berechnung dieses Wertes erfolgt dann im Methodenrumpf. Das Schlüsselwort `return` gibt an, dass nun der angesprochene Rückgabewert retourniert wird.

5.5.1. Ausgabe, Console.WriteLine

Mittels `Console.WriteLine` können Ausgaben auf die Konsole gemacht werden. Berechnungen und Ergebnisse können somit dem Benutzer (Anwender) des Programmes angezeigt werden. Zur Ausgabe von Objekten erstellen wir jeweils eigene Methoden oder überschreiben `ToString()`.

```
public class Student
{
    private string _name;
    private int _geburtsjahr;

    public override string ToString()
    {
        return $"Name: {_name} (Geburtsjahr: {_geburtsjahr})";
    }

    public void Ausgeben()
    {
        Console.WriteLine(this);
    }
}
```

Das `$"..."` ist eine interpolated string literal, die es ermöglicht, Variablen direkt in Strings einzubetten.

5.6. Signatur von Methoden

Die Signatur einer Methode ist gegeben durch den Methodennamen, sowie Typen und Reihenfolge der Parameter. In einer Klasse dürfen nicht mehrere Methoden mit der selben Signatur vorkommen. Mehrere Methoden mit gleichem Namen sind sehr wohl zulässig, da sie eindeutig durch die Parameter unterschieden werden können. Man nennt dies auch *Überladen* von Methoden.

Wichtig: Der Rückgabewert gehört *nicht* zur Signatur!

```
public class SignaturDemo
{
    public int Foo(int a, int b) { return a + b; }

    public int Foo(int a) { return a * 2; } // Zulässig: andere Parameteranzahl

    public double Foo(double a, double b) { return a + b; } // Zulässig: andere Parametertypen

    /*
    public double Foo(int x) { return x * 1.0; }
    UNZULÄSSIG: Es existiert bereits eine Methode Foo(int).
    Der unterschiedliche Rückgabewert (double vs int) reicht zur Unterscheidung nicht aus!
    */
}

public class Berechnung
{
    public int Summe(int a, int b)
    {
        return a + b;
    }
}
```

```
}

public double Summe(double a, double b)
{
    return a + b;
}
}
```

Hier ist es sinnvoll, die Methode zur Summenbildung von `int`-Parametern sowie `double`-Parametern gleich zu benennen, also zu überladen.

5.7. Parameter-Prüfungen (Fail-Fast)

Wir betrachten in diesem Abschnitt die Klasse `Person` mit Fail-Fast-Validierung:

```
public class Person
{
    private string _name;
    private bool _maennlich;
    private int _groesse;
    private int _gewicht;

    public Person(string name, bool maennlich, int groesse, int gewicht)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));
        if (groesse < 110 || groesse > 220)
            throw new ArgumentOutOfRangeException(nameof(groesse),
                "Größe muss zwischen 110 und 220 cm liegen");
        if (gewicht < 30 || gewicht > 150)
            throw new ArgumentOutOfRangeException(nameof(gewicht),
                "Gewicht muss zwischen 30 und 150 kg liegen");

        _name = name;
        _maennlich = maennlich;
        _groesse = groesse;
        _gewicht = gewicht;
    }

    public string Name => _name;
    public bool Maennlich => _maennlich;
    public int Groesse => _groesse;
    public int Gewicht => _gewicht;

    public override string ToString()
    {
        string geschlecht = _maennlich ? "m" : "f";
        return $"Name: {_name} ({geschlecht}), Größe: {_groesse}, Gewicht: {_gewicht}";
    }
}
```

Im Gegensatz zum Happy-Path-Pattern werden hier bei ungültigen Parametern sofort Exceptions geworfen. Der Aufrufer muss sich um die Fehlerbehandlung kümmern, was zu robusterem Code führt.

5.8. Lokale Variablen, Berechnungen und Typkonvertierung

Wir betrachten nun die folgende Aufgabenstellung: Berechnung des BMI (Body-Mass-Index).

```
public class Person
{
    // ... Felder und Properties wie oben ...

    public double BerechneBmi()
    {
        double groesseMeter = _groesse / 100.0;
        return _gewicht / (groesseMeter * groesseMeter);
    }

    public void BmiAusgeben()
    {
        double bmi = BerechneBmi();
        string bmiTyp;

        if (_maennlich)
        {
            if (bmi < 20)
                bmiTyp = "Untergewicht";
            else if (bmi <= 25)
                bmiTyp = "Normalgewicht";
            else
                bmiTyp = "Übergewicht";
        }
        else
        {
            if (bmi < 19)
                bmiTyp = "Untergewicht";
            else if (bmi <= 24)
                bmiTyp = "Normalgewicht";
            else
                bmiTyp = "Übergewicht";
        }

        Console.WriteLine($"BMI: {bmi:F2} - {bmiTyp}");
    }
}
```

In der Methode `BerechneBmi` wird zunächst die Größe von Zentimeter in Meter umgerechnet. Beachten Sie, dass wir `100.0` verwenden, um eine Fließkommadivision zu erzwingen. Die Formatangabe `F2` in `Console.WriteLine` gibt an, dass der Wert mit 2 Nachkommastellen formatiert werden soll.

Kapitel 6. Variablen, Datentypen und Methoden

Im letzten Abschnitt wurden bereits die Datentypen `string` und `int` verwendet. Diese fanden Anwendung bei *Feldern* und *Parametern*. Der Oberbegriff dafür lautet **Variablen**, wir werden später noch eine weitere Art von Variablen, nämlich *lokale Variablen* kennen lernen.

Der Datentyp legt fest, welche Art von Werten bei einer Variablen hinterlegt sind. Wir haben bereits `int` als Datentyp für ganze Zahlen kennengelernt, und `string` als Datentyp für Zeichenketten. Wir betrachten zunächst die sogenannten *Werttypen* genauer.

6.1. Werttypen

Die Werttypen (value types) sind in der Programmiersprache C# "fest eingebaut". Die wichtigsten Werttypen umfassen:

Typ	Bereich	Verwendung
<code>byte</code>	0 bis 255	Kleine positive Ganzzahlen
<code>int</code>	-2.147.483.648 bis 2.147.483.647	Ganze Zahlen (Standard)
<code>long</code>	-9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807	Sehr große Ganzzahlen
<code>float</code>	$\pm 1,5 \times 10^{-45}$ bis $\pm 3,4 \times 10^{38}$	Kommazahlen (präzise)
<code>double</code>	$\pm 5,0 \times 10^{-324}$ bis $\pm 1,7 \times 10^{308}$	Kommazahlen (Standard)
<code>decimal</code>	$\pm 1,0 \times 10^{-28}$ bis $\pm 7,9 \times 10^{28}$	Finanz- und monetäre Berechnungen
<code>bool</code>	true oder false	Logische Wahrheitswerte
<code>char</code>	Unicode-Zeichen	Einzelnes Zeichen
<code>DateTime</code>	01.01.0001 bis 31.12.9999	Datum und Uhrzeit

Für ganze Zahlen ist meist `int` die beste Wahl, nur wenn extrem große Zahlen dargestellt werden sollen muss `long` verwendet werden. Für Kommazahlen wird meist `double` verwendet. Für finanzielle Berechnungen ist `decimal` zu bevorzugen, da es keine Rundungsfehler hat.

6.2. Referenztypen

Im vorigen Kapitel haben wir die Klasse `Student` entwickelt, und damit einen eigenen *Datentyp* erstellt! *Alle* Klassen definieren einen Datentyp, im Gegensatz zu den Werttypen, die ja ein integraler Bestandteil der Programmiersprache sind, handelt es sich hierbei um einen **Referenztyp**. Auch beim schon verwendeten Typ `string` handelt es sich um einen derartigen Referenztyp.

Ein wesentlicher Unterschied von Referenztypen zu Werttypen ist, dass sie nicht den eigentlichen *Wert* selbst enthalten, sondern eine *Referenz* (in anderen Worten: einen Verweis)

auf den Speicherort, an dem die Daten tatsächlich gespeichert sind.

Daraus ergeben sich die folgenden Konsequenzen für den Umgang mit Referenztypen:

- Der Verweis kann auch "ins Leere" gehen, also auf nichts Eigentliches verweisen. Der Wert der Objektreferenz ist dann `null`. **Ist der Wert einer Objektreferenz ``null``, so können darauf keine Methoden aufgerufen werden. Dies würde zu einer `NullReferenceException` führen!**
- Umgekehrt ist es möglich, dass auf ein und den selben Inhalt (im Speicher) **mehrere Referenzen** existieren. Es kann also eine Variable `stud1` auf den selben Studenten wie die Variable `stud2` verweisen!

6.3. Methoden (Fortsetzung)

Wir haben schon spezielle Methoden kennengelernt, nämlich Properties und Konstruktoren. Methoden sind also zusammengehörige Code-Stücke, die Teil einer Klasse sind. Eine Methode setzt sich aus dem *Methodenkopf* und dem *Methodenrumpf* zusammen. Der Methodenkopf enthält einen Zugriffs-Modifizier (`private` oder `public`), einen Rückgabewert, einen Methoden-Namen, sowie eine möglicherweise auch leere Parameterliste.

Der Zugriffsmodifizier steuert die "*Sichtbarkeit*" der Methode. Alle `private` gesetzten Elemente sind außerhalb der Klasse *nicht* sichtbar, d.h. sie können von dort aus nicht aufgerufen werden. Hingegen bedeutet `public`, dass das Element überall sichtbar ist.

Der **Rückgabewert** einer Methode ist entweder ein zurückgegebener Wert, oder `void` für Methoden, die nichts zurückgeben.

In den runden Klammern sind die Parameter einer Methode angegeben. Standardmäßig werden in C# alle Parameter als **Kopie** übergeben (**Call-by-Value**):

- Bei **Werttypen** (z. B. `int`, `double`, `bool`) wird der eigentliche Wert kopiert. Änderungen am Parameter innerhalb der Methode haben **keine Auswirkungen** auf die ursprüngliche Variable des Aufrufers.
- Bei **Referenztypen** (z. B. Klassen, Strings) wird die Referenz (der Verweis) kopiert. Da die kopierte Referenz auf dasselbe Objekt im Speicher zeigt, wirken sich Änderungen am **Zustand** des Objekts (z. B. Ändern eines Feldes) direkt aus. Die Variable des Aufrufers kann jedoch nicht innerhalb der Methode auf ein völlig neues Objekt umgebogen werden.

```
public class Student
{
    private string _name;
    private int _geburtsjahr;

    public int BerechneAlter(int aktuellesJahr)
    {
        // Parameter 'aktuellesJahr' ist eine Kopie des Wertes
        return aktuellesJahr - _geburtsjahr;
    }
}
```

6.4. Parameterübergabe mit `ref` und `out`

Möchte man eine Methode in die Lage versetzen, die Variable des Aufrufers direkt zu verändern (Call-by-Reference), nutzt man die Schlüsselwörter `ref` oder `out`.

Das Schlüsselwort `ref` Wird ein Parameter mit `ref` markiert, wird keine Kopie erstellt, sondern eine Referenz auf die Variable selbst übergeben. Die Variable muss **vor** dem Aufruf initialisiert sein.

```
public void Verdopple(ref int zahl)
{
    zahl = zahl * 2; // Ändert die Variable des Aufrufers direkt
}

// Aufruf:
int meinWert = 10;
Verdopple(ref meinWert);
// meinWert ist nun 20
```

Das Schlüsselwort `out` `out` funktioniert ähnlich wie `ref`, wird aber verwendet, wenn eine Methode einen Wert "hinausgeben" soll. Die Variable muss vor dem Aufruf **nicht** initialisiert sein, aber die Methode **muss** ihr vor dem Ende einen Wert zuweisen.

```
public void BerechneWerte(int a, int b, out int summe, out int produkt)
{
    summe = a + b;
    produkt = a * b;
}

// Aufruf:
BerechneWerte(10, 5, out int s, out int p);
// s = 15, p = 50
```

6.5. Wann ist die Verwendung von `ref` und `out` sinnvoll?

Die Verwendung von `ref` und `out` sollte sparsam erfolgen, da sie den Code schwerer lesbar machen können. Es gibt jedoch klare Anwendungsfälle:

- **Das Try-Pattern (für `out`):** Dies ist der häufigste Fall in der .NET-Entwicklung. Viele Methoden geben einen `bool` zurück, um Erfolg zu signalisieren, und den eigentlichen Wert über `out`. Das bekannteste Beispiel ist `int.TryParse`.
- **Rückgabe mehrerer Werte:** Wenn eine Methode mehr als ein Ergebnis liefern muss. Obwohl man hier heutzutage oft **Tuples** bevorzugt, ist `out` eine bewährte Methode.
- **Performance (für `ref`):** Bei extrem großen Werttypen (structs), um das Kopieren des gesamten Objekts zu vermeiden. Da wir in der Regel mit Klassen (`class`) arbeiten, die ohnehin als Referenz übergeben werden, ist dies selten notwendig.

Best Practice: Nutzen Sie für normale Berechnungen immer den Rückgabewert (`return`). Verwenden Sie `out` primär für das Try-Pattern und vermeiden Sie `ref`, sofern es keine zwingenden technischen Gründe gibt.

Kapitel 7. Objekt-Beziehungen

Oftmals stehen Objekte in Beziehung zueinander. Ein häufiger Fall ist, dass in Objekten einer Klasse mehrere Instanzen einer anderen Klasse verwaltet werden. Im vorigen Abschnitt wurde die Klasse `Person` entwickelt. In diesem Abschnitt wird eine Klasse `Auto` entwickelt, in das bis zu drei Personen "einsteigen" können.

Die Plätze im Auto werden durch Felder, konkret durch Referenzen auf Objekte vom Typ `Person` umgesetzt. Steigt eine konkrete Person in das Auto ein, so wird im Objekt des Typs `Auto` eine Objektreferenz auf das Objekt der Person gesetzt.

```
public class Auto
{
    private string _name;
    private int _eigengewicht;

    private Person _fahrer;
    private Person _beifahrer;
    private Person _rueckbank;

    public Auto(string name, int eigengewicht)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));
        if (eigengewicht < 600 || eigengewicht > 3000)
            throw new ArgumentOutOfRangeException(nameof(eigengewicht));

        _name = name;
        _eigengewicht = eigengewicht;
    }

    public string Name => _name;
    public int Eigengewicht => _eigengewicht;

    public void Einsteigen(Person person)
    {
        if (person == null)
            throw new ArgumentNullException(nameof(person));

        if (_fahrer == null)
            _fahrer = person;
        else if (_beifahrer == null)
            _beifahrer = person;
        else if (_rueckbank == null)
            _rueckbank = person;
        else
            throw new InvalidOperationException("Das Auto ist voll!");
    }

    public void Aussteigen(Person person)
    {
        if (person == null)
            throw new ArgumentNullException(nameof(person));

        if (_fahrer == person)
            _fahrer = null;
        else if (_beifahrer == person)
```



```

        _beifahrer = null;
    else if (_rueckbank == person)
        _rueckbank = null;
    else
        throw new InvalidOperationException("Die Person sitzt nicht im Auto.");
}

public void Aussteigen(string name)
{
    if (string.IsNullOrEmpty(name))
        throw new ArgumentException("Name darf nicht leer sein", nameof(name));

    // In C# können Strings einfach mit == verglichen werden
    if (_fahrer != null && _fahrer.Name == name)
        _fahrer = null;
    else if (_beifahrer != null && _beifahrer.Name == name)
        _beifahrer = null;
    else if (_rueckbank != null && _rueckbank.Name == name)
        _rueckbank = null;
    else
        throw new InvalidOperationException($"Person '{name}' nicht im Auto gefunden.");
}

public int Gesamtgewicht()
{
    int gewicht = _eigengewicht;

    if (_fahrer != null)
        gewicht += _fahrer.Gewicht;
    if (_beifahrer != null)
        gewicht += _beifahrer.Gewicht;
    if (_rueckbank != null)
        gewicht += _rueckbank.Gewicht;

    return gewicht;
}

public override string ToString()
{
    var sb = new System.Text.StringBuilder();
    sb.AppendLine($"Auto: {_name}, Eigengewicht: {_eigengewicht}");
    sb.AppendLine(new string('-', 50));
    sb.AppendLine($"Fahrer: {_fahrer?.ToString()} ?? "--frei--");
    sb.AppendLine($"Beifahrer: {_beifahrer?.ToString()} ?? "--frei--");
    sb.AppendLine($"Rückbank: {_rueckbank?.ToString()} ?? "--frei--");
    sb.AppendLine(new string('-', 50));

    return sb.ToString();
}
}

```

Die wesentlichen Überlegungen zu derartigen Aufgaben finden sich in der Methode `Einsteigen`. Diese Methode hat einen Parameter vom Typ `Person` mittels dem die Objektreferenz auf die einsteigende Person übergeben wird. Die zugrundeliegende Logik ist: die Person nimmt den ersten freien Platz. Die Methode verwendet das Fail-Fast-Pattern und wirft eine Exception wenn das Auto voll ist.

Beachten Sie im `ToString()`-Method die Verwendung des *null-conditional operators* (`?.`) und des *null-coalescing operators* (`??`). Diese ermöglichen eine kompakte und sichere Handhabung von potentiell null-Werten.

Kapitel 8. Strings

Der Typ `string` ist ein integraler Bestandteil von C# und enthält zahlreiche nützliche Methoden zur Bearbeitung und Analyse von Zeichenketten. Strings sind in C# unveränderlich (immutable) - jede Modifikation erzeugt einen neuen String.

Wichtige Methoden der Klasse `string`:

- `char this[int index]`: Gibt das Zeichen an der Stelle `index` zurück (erste Stelle hat Index 0!)
- `int IndexOf(string str)`: Gibt die Stelle des ersten Vorkommens des übergebenen Strings zurück
- `bool IsNullOrEmpty(string)`: Statische Methode, prüft ob der String null oder leer ist
- `bool IsNullOrWhiteSpace(string)`: Statische Methode, prüft auf null, leer oder nur Leerzeichen
- `int Length`: Gibt die Länge des Strings zurück
- `string ToLower()`: Retourniert den String in Kleinbuchstaben
- `string ToUpper()`: Retourniert den String in Großbuchstaben
- `string Trim()`: Entfernt Leerzeichen am Anfang und Ende
- `string[] Split(char[] separator)`: Teilt den String an den angegebenen Trennzeichen
- `bool StartsWith(string)`: Prüft ob der String mit dem angegebenen Text beginnt
- `bool EndsWith(string)`: Prüft ob der String mit dem angegebenen Text endet

```
public class Person
{
    private string _vorname;
    private string _nachname;

    public Person(string vorname, string nachname)
    {
        if (string.IsNullOrEmpty(vorname))
            throw new ArgumentException("Vorname darf nicht leer sein", nameof(vorname));
        if (string.IsNullOrEmpty(nachname))
            throw new ArgumentException("Nachname darf nicht leer sein", nameof(nachname));

        _vorname = vorname;
        _nachname = nachname;
    }

    public string VollerName => $"{_vorname} {_nachname}";

    public int LaengeVollerName => VollerName.Length;

    public string Initialen
    {
        get
        {
            if (_vorname.Length == 0 || _nachname.Length == 0)
                throw new InvalidOperationException("Vor- oder Nachname ist leer");

            return $"{_vorname[0]}. {_nachname[0]}.";
        }
    }
}
```

```
public string Kuerzel
{
    get
    {
        if (_nachname.Length < 2 || _vorname.Length < 1)
            throw new InvalidOperationException("Name zu kurz für Kürzel");

        return $"{_nachname.Substring(0, 2).ToUpper()}{_vorname[0]}";
    }
}
```

8.1. String-Vergleich

Möchte man zwei Strings auf Inhaltsgleichheit überprüfen, sollte die `==`- oder `!=`-Operatoren verwendet werden (im Gegensatz zu Java, wo `equals()` benötigt wird). In C# ist der `==`-Operator für Strings überladen und vergleicht den Inhalt:

```
string a = "xyz";
string b = "xy" + "z";

// Beide Vergleiche funktionieren korrekt in C#:
Console.WriteLine(a == b);           // true (Inhaltsvergleich)
Console.WriteLine(a.Equals(b));      // true (expliziter Methodenaufruf)
```

8.2. StringBuilder

Da Strings in C# unveränderlich (immutable) sind, erzeugt jede Änderung (z.B. mit `+`) ein neues String-Objekt im Speicher. Bei vielen Änderungen in einer Schleife kann dies die Performance beeinträchtigen. In solchen Fällen verwendet man den `StringBuilder`.

```
using System.Text;

var sb = new StringBuilder();
sb.Append("Start");
for (int i = 0; i < 100; i++)
{
    sb.Append(i);
    sb.Append(", ");
}
string ergebnis = sb.ToString();
```

Kapitel 9. Schleifen

Schleifen sind zentrale Sprachkonstrukte in allen Programmiersprachen. Sie ermöglichen es bestimmte Aufgaben mehrmals durchzuführen.

9.1. `while`-Schleife

```
public void ZeichneLinie(int laenge)
{
    if (laenge < 0)
        throw new ArgumentOutOfRangeException(nameof(laenge));

    int zaehler = 0;
    while (zaehler < laenge)
    {
        Console.Write("-");
        zaehler++;
    }
    Console.WriteLine();
}
```

9.2. `do-while`-Schleife

```
public void ZahlenAusgeben()
{
    int i = 0;
    do
    {
        Console.WriteLine(i);
        i++;
    } while (i < 10);
}
```

9.3. `for`-Schleife

```
public void ZahlenAusgeben2()
{
    for (int i = 0; i < 10; i++)
    {
        Console.WriteLine(i);
    }
}
```

9.4. `foreach`-Schleife

Für die Iteration über Collections ist die `foreach`-Schleife besonders elegant:

```
public void NamenAusgeben(List<string> namen)
{
    foreach (string name in namen)
    {
        Console.WriteLine(name);
    }
}
```

9.5. Codebeispiele und Code-Wiederverwendung

Beim Programmieren sollte man stets versuchen, bestimmte Teilaufgaben nicht mehrfach zu

implementieren. Das Prinzip der Code-Wiederverwendung besagt, dass schon implementierte Codestücke (in Methoden) nach Möglichkeit überall dort wiederverwendet werden sollen, wo sie als Teilaufgabe einer anderen Aufgabe auftreten.

9.5.1. Beispiel: Quadrat mit Diagonale

In diesem Beispiel soll eine Klasse `Quadrat` entwickelt werden, die ein Quadrat mit einer Diagonale von links oben nach rechts unten auf der Konsole ausgeben kann.

```
public class Quadrat
{
    private int _laenge;

    public Quadrat(int laenge)
    {
        if (laenge <= 0 || laenge >= 100)
            throw new ArgumentOutOfRangeException(nameof(laenge), "Länge muss zwischen 1 und 99 liegen");

        _laenge = laenge;
    }

    public void Ausgeben()
    {
        for (int zeile = 0; zeile < _laenge; zeile++)
        {
            for (int spalte = 0; spalte < _laenge; spalte++)
            {
                if (zeile == 0 || zeile == _laenge - 1 ||
                    spalte == 0 || spalte == _laenge - 1 ||
                    zeile == spalte)
                {
                    Console.Write("#");
                }
                else
                {
                    Console.Write(" ");
                }
            }
            Console.WriteLine();
        }
    }

    /// <summary>
    /// Zeichnet das Quadrat mit der anderen Diagonale (rechts oben nach links unten)
    /// </summary>
    public void AusgebenInvers()
    {
        for (int zeile = 0; zeile < _laenge; zeile++)
        {
            for (int spalte = 0; spalte < _laenge; spalte++)
            {
                if (zeile == 0 || zeile == _laenge - 1 ||
                    spalte == 0 || spalte == _laenge - 1)
                {
                    Console.Write("#");
                }
            }
        }
    }
}
```

```

        else if (spalte == _laenge - 1 - zeile)
        {
            Console.Write("*");
        }
        else
        {
            Console.Write(" ");
        }
    }
    Console.WriteLine();
}
}
}

```

9.5.2. Beispiel: Rechteck mit Code-Wiederverwendung

Das folgende Beispiel demonstriert, wie man eine komplexe Aufgabe (Rechteck zeichnen) in kleinere Teilaufgaben (Linie zeichnen) zerlegt.

```

public class SchleifenDemo
{
    public void ZeichneLinie(int einrueckung, int laenge)
    {
        ZeichneLinie(einrueckung, laenge, '*', '#');
    }

    public void ZeichneLinie(int einrueckung, int laenge, char fuellZeichen, char randZeichen)
    {
        if (einrueckung < 0)
            throw new ArgumentOutOfRangeException(nameof(einrueckung), "Einrückung darf nicht negativ sein");
        if (laenge < 0)
            throw new ArgumentOutOfRangeException(nameof(laenge), "Länge darf nicht negativ sein");

        // Einrückung zeichnen
        for (int i = 0; i < einrueckung; i++)
        {
            Console.Write(" ");
        }

        // Linie zeichnen
        for (int i = 0; i < laenge; i++)
        {
            if (i == 0 || i == laenge - 1)
            {
                Console.Write(randZeichen + " ");
            }
            else
            {
                Console.Write(fuellZeichen + " ");
            }
        }
        Console.WriteLine();
    }

    public void ZeichneRechteck(int einrueckung, int laenge, int hoehe, char fuellZeichen, char randZeichen)
    {

```

```
if (hoehe < 0)
    throw new ArgumentOutOfRangeException(nameof(hoehe));

for (int zeile = 0; zeile < hoehe; zeile++)
{
    if (zeile == 0 || zeile == hoehe - 1)
    {
        // Erste und letzte Zeile verwenden das Randzeichen überall
        ZeichneLinie(einrueckung, laenge, randZeichen, randZeichen);
    }
    else
    {
        // Mittlere Zeilen verwenden das Füllzeichen
        ZeichneLinie(einrueckung, laenge, fuellZeichen, randZeichen);
    }
}

public void ZeichneQuadrat(int einrueckung, int laenge)
{
    // Ein Quadrat ist ein Spezialfall eines Rechtecks
    ZeichneRechteck(einrueckung, laenge, laenge, '.', '#');
}
}
```


Kapitel 10. Arrays und Collections

10.1. Arrays

Ein Array ist ein "Feld" oder eine Anordnung mehrerer Werte des selben Datentyps, wobei die konkreten Elemente des Arrays mittels *Index* angesprochen werden.

```
// Deklaration und Initialisierung
int[] zahlen = new int[10];

// Zuweisung
zahlen[0] = 42;
zahlen[1] = 23;

// Auslesen
int ersteZahl = zahlen[0];

// Länge ermitteln
int anzahl = zahlen.Length;

// Iteration
for (int i = 0; i < zahlen.Length; i++)
{
    Console.WriteLine(zahlen[i]);
}
```

10.2. Generische Listen (List<T>)

Im Gegensatz zu Arrays, deren Größe fix ist, können `List<T>` dynamisch wachsen:

```
using System.Collections.Generic;

// Erstellen
List<string> namen = new List<string>();

// Hinzufügen
namen.Add("Max");
namen.Add("Anna");

// Zugriff (wie bei Array)
string ersterName = namen[0];

// Anzahl Elemente
int anzahl = namen.Count;

// Entfernen
namen.RemoveAt(0); // Entfernt erstes Element
namen.Remove("Max"); // Entfernt nach Wert

// Iteration
foreach (string name in namen)
{
    Console.WriteLine(name);
}
```

10.3. Weitere Collection-Typen

Typ	Beschreibung
<code>Dictionary<K, V></code>	Schlüssel-Wert-Paare mit schnellem Zugriff
<code>HashSet<T></code>	Unsortierte Menge ohne Duplikate
<code>Queue<T></code>	FIFO-Warteschlange (First In, First Out)
<code>Stack<T></code>	LIFO-Stapel (Last In, First Out)

10.4. Manuelle Array-Verwaltung

In der professionellen Entwicklung verwenden wir meist `List<T>`. Aus didaktischen Gründen ist es jedoch wichtig zu verstehen, wie man die Größe eines Arrays manuell verwaltet. Da ein Array eine fixe Größe hat, müssen wir oft selbst mitzählen, wie viele "echte" Daten bereits darin gespeichert sind.

```
public class Messreihe
{
    private double[] _temperaturen;
    private int _anzahl; // Zähler für die aktuell belegten Plätze

    public Messreihe(int maximaleKapazitaet)
    {
        if (maximaleKapazitaet <= 0)
            throw new ArgumentException("Kapazität muss positiv sein");

        _temperaturen = new double[maximaleKapazitaet];
        _anzahl = 0;
    }

    public void Hinzufuegen(double temperatur)
    {
        // Fail-Fast: Prüfen ob noch Platz ist
        if (_anzahl >= _temperaturen.Length)
            throw new InvalidOperationException("Die Messreihe ist voll!");

        _temperaturen[_anzahl] = temperatur;
        _anzahl++;
    }

    public void Einfuegen(int index, double temperatur)
    {
        if (index < 0 || index > _anzahl)
            throw new ArgumentOutOfRangeException(nameof(index));
        if (_anzahl >= _temperaturen.Length)
            throw new InvalidOperationException("Kein Platz zum Einfügen");

        // Alle Elemente nach hinten verschieben (von hinten beginnend!)
        for (int i = _anzahl; i > index; i--)
        {
            _temperaturen[i] = _temperaturen[i - 1];
        }

        _temperaturen[index] = temperatur;
        _anzahl++;
    }

    public void Entfernen(int index)
```

```
{
    if (index < 0 || index >= _anzahl)
        throw new ArgumentOutOfRangeException(nameof(index));

    // Alle Elemente nach vorne verschieben
    for (int i = index; i < _anzahl - 1; i++)
    {
        _temperaturen[i] = _temperaturen[i + 1];
    }

    _anzahl--;
    _temperaturen[_anzahl] = 0; // Optional: Letzten Platz "leeren"
}

public double BerechneDurchschnitt()
{
    if (_anzahl == 0) return 0;

    double summe = 0;
    for (int i = 0; i < _anzahl; i++)
    {
        summe += _temperaturen[i];
    }
    return summe / _anzahl;
}
```

Kapitel 11. Statische Methoden und Nützliche Klassen

11.1. Statische Methoden

Manchmal möchte man eine Methode verwenden, ohne vorher ein Objekt der Klasse zu erzeugen. Dies ist sinnvoll, wenn die Methode keinerlei Informationen aus dem Zustand eines Objektes benötigt (keine Felder verwendet). Solche Methoden werden mit dem Schlüsselwort `static` markiert.

```
public class Kurs
{
    // Statische Methode: Gehört zur Klasse, nicht zum Objekt
    public static bool IstZulassungMoeglich(Student student)
    {
        if (student == null) return false;

        // Zulassung nur wenn Geburtsjahr vor 2005
        return student.Geburtsjahr < 2005;
    }
}

// Aufruf ohne Instanz:
bool darfTeilnehmen = Kurs.IstZulassungMoeglich(meinStudent);
```

11.2. Die `Main`-Methode

Einen wichtigen Spezialfall einer statischen Methode stellt die `Main`-Methode dar. Sie definiert den Einstiegspunkt in ein Programm. Wenn eine Anwendung gestartet wird, sucht das Betriebssystem nach dieser Methode.

```
public class Programm
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Programm gestartet!");
        // Hier beginnt die Ausführung
    }
}
```

In modernen C#-Versionen (.NET 5+) können auch **Top-Level Statements** verwendet werden. Dabei schreibt man den Code direkt in die Datei `Program.cs` ohne explizite `Main`-Methode. Der Compiler erzeugt die `Main`-Methode dann automatisch im Hintergrund.

11.3. Nützliche Klassen: `Math` und `Random`

11.3.1. Die Klasse `Math`

Die Klasse `Math` bietet zahlreiche statische Methoden für mathematische Berechnungen:

- `Math.Abs(x)`: Absolutbetrag
- `Math.Sqrt(x)`: Quadratwurzel
- `Math.Pow(x, y)`: Potenz (x^y)
- `Math.Round(x)`: Runden
- `Math.Max(a, b)` / `Math.Min(a, b)`: Maximum/Minimum

11.3.2. Die Klasse `Random`

Für die Erzeugung von Zufallszahlen verwenden wir die Klasse `Random`. Im Gegensatz zu Java erzeugt man in C# meist eine Instanz und verwendet diese wieder:

```
Random wuerfel = new Random();

// Zufallszahl zwischen 1 (inklusive) und 7 (exklusive) -> 1 bis 6
int ergebnis = wuerfel.Next(1, 7);

// Zufällige Kommazahl zwischen 0.0 und 1.0
double wahrscheinlichkeit = wuerfel.NextDouble();
```

Kapitel 12. Vererbung

Vererbung ist ein fundamentales Konzept der objektorientierten Programmierung. Sie ermöglicht es, eine neue Klasse auf Basis einer bestehenden Klasse zu erstellen. Die abgeleitete Klasse erbt dabei alle Eigenschaften und Methoden der Basisklasse und kann diese erweitern oder anpassen.

12.1. Grundlagen der Vererbung

In C# wird Vererbung mit dem Doppelpunkt-Operator (:) ausgedrückt. Eine Klasse kann nur von **einer** Basisklasse erben (Einfachvererbung).

```
// Basisklasse (auch Oberklasse oder Superklasse genannt)
public class Person
{
    private string _name;
    private int _geburtsjahr;

    public Person(string name, int geburtsjahr)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));
        if (geburtsjahr < 1900 || geburtsjahr > DateTime.Now.Year)
            throw new ArgumentOutOfRangeException(nameof(geburtsjahr));

        _name = name;
        _geburtsjahr = geburtsjahr;
    }

    public string Name => _name;
    public int Geburtsjahr => _geburtsjahr;

    public virtual int BerechneAlter(int aktuellesJahr)
    {
        if (aktuellesJahr < _geburtsjahr)
            throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

        return aktuellesJahr - _geburtsjahr;
    }

    public override string ToString()
    {
        return $"{_name} (geb. {_geburtsjahr})";
    }
}

// Abgeleitete Klasse (auch Unterklasse oder Subklasse genannt)
public class Student : Person
{
    private string _matrikelnummer;
    private int _semester;

    public Student(string name, int geburtsjahr, string matrikelnummer, int semester = 1)
        : base(name, geburtsjahr) // Aufruf des Basisklassen-Konstruktors
    {
        if (string.IsNullOrEmpty(matrikelnummer))
            throw new ArgumentException("Matrikelnummer darf nicht leer sein", nameof
```

```
(matrikelnummer));
    if (semester < 1 || semester > 20)
        throw new ArgumentOutOfRangeException(nameof(semester), "Semester muss zwischen 1 und 20 liegen");

    _matrikelnummer = matrikelnummer;
    _semester = semester;
}

public string Matrikelnummer => _matrikelnummer;
public int Semester => _semester;

public override string ToString()
{
    return $"{base.ToString()} - {_matrikelnummer}, {_semester}. Semester";
}
}
```

Wichtige Schlüsselwörter:

- `virtual`: Markiert eine Methode in der Basisklasse, die überschrieben werden *darf*
- `override`: Überschreibt eine virtuelle oder abstrakte Methode der Basisklasse
- `base`: Greift auf Member der Basisklasse zu (z.B. `base.ToString()`)

12.2. Der Zugriffsmodifizierer `protected`

Zusätzlich zu `public` und `private` gibt es `protected`. Ein `protected`-Member ist: * In der eigenen Klasse sichtbar * In allen abgeleiteten Klassen sichtbar * Von außen **nicht** sichtbar

```
public class Fahrzeug
{
    private string _seriennummer; // Nur in Fahrzeug sichtbar
    protected int _kilometerstand; // In Fahrzeug und abgeleiteten Klassen sichtbar
    public string Marke { get; } // Überall sichtbar

    public Fahrzeug(string marke, string seriennummer)
    {
        Marke = marke;
        _seriennummer = seriennummer;
        _kilometerstand = 0;
    }

    public void Fahren(int kilometer)
    {
        if (kilometer < 0)
            throw new ArgumentException("Kilometer darf nicht negativ sein");
        _kilometerstand += kilometer;
    }
}

public class Elektroauto : Fahrzeug
{
    private int _batteriestand; // in Prozent

    public Elektroauto(string marke, string seriennummer, int batteriestand)
        : base(marke, seriennummer)
    {
    }
}
```

```

{
    if (batteriestand < 0 || batteriestand > 100)
        throw new ArgumentOutOfRangeException(nameof(batteriestand));
    _batteriestand = batteriestand;
}

public int Batteriestand => _batteriestand;

public void Laden(int prozent)
{
    if (prozent < 0)
        throw new ArgumentException("Ladung darf nicht negativ sein");

    _batteriestand = Math.Min(100, _batteriestand + prozent);

    // _kilometerstand ist hier zugreifbar wegen protected!
    Console.WriteLine($"Fahrzeug mit {_kilometerstand} km geladen.");
}

// _seriennummer ist hier NICHT zugreifbar (private in Basisklasse)
}

```

Faustregel für Zugriffsmodifizierer:

Modifizierer	Sichtbarkeit
<code>public</code>	Überall sichtbar
<code>protected</code>	In der Klasse und allen abgeleiteten Klassen
<code>private</code>	Nur in der eigenen Klasse
<code>internal</code>	Im gesamten Assembly (Projekt)

12.3. Methoden überschreiben vs. Überladen

Es ist wichtig, zwischen dem **Überschreiben** (`override`) und dem **Überladen** (Overloading) zu unterscheiden:

```

public class Tier
{
    public virtual void GibLaut()
    {
        Console.WriteLine("Das Tier macht ein Geräusch");
    }

    public void GibLaut(int anzahl)
    {
        for (int i = 0; i < anzahl; i++)
        {
            GibLaut();
        }
    }
}

public class Hund : Tier
{
    public override void GibLaut() // Überschreiben: gleiche Signatur
    {

```



```
        Console.WriteLine("Wuff!");
    }

    // GibLaut(int) wird geerbt und kann ohne Neuimplementierung verwendet werden
}

// Verwendung:
Hund hund = new Hund();
hund.GibLaut();        // "Wuff!" - überschriebene Methode
hund.GibLaut(3);       // "Wuff!" 3x - geerbte überladene Methode
```

12.4. Polymorphie in der Praxis

Polymorphie ermöglicht es, Objekte unterschiedlicher abgeleiteter Klassen einheitlich zu behandeln. Eine Variable vom Typ der Basisklasse kann auf Objekte beliebiger abgeleiteter Klassen verweisen.

```
public abstract class Shape
{
    public abstract double BerechneFlaeche();
    public abstract double BerechneUmfang();
}

public class Rechteck : Shape
{
    private double _breite;
    private double _hoehe;

    public Rechteck(double breite, double hoehe)
    {
        if (breite <= 0 || hoehe <= 0)
            throw new ArgumentException("Seiten müssen positiv sein");
        _breite = breite;
        _hoehe = hoehe;
    }

    public override double BerechneFlaeche() => _breite * _hoehe;
    public override double BerechneUmfang() => 2 * (_breite + _hoehe);
}

public class Kreis : Shape
{
    private double _radius;

    public Kreis(double radius)
    {
        if (radius <= 0)
            throw new ArgumentException("Radius muss positiv sein");
        _radius = radius;
    }

    public override double BerechneFlaeche() => Math.PI * _radius * _radius;
    public override double BerechneUmfang() => 2 * Math.PI * _radius;
}

// Polymorphe Verwendung:
List<Shape> formen = new List<Shape>
{
```

```
new Rechteck(3, 4),
new Kreis(5),
new Rechteck(2, 6),
new Kreis(2.5)
};

foreach (Shape form in formen)
{
    Console.WriteLine($"Fläche: {form.BerechneFlaeche():F2}, Umfang: {form.BerechneUmfang():F2}");
}
```

Der Aufrufer muss nicht wissen, um welche konkrete Form es sich handelt - er arbeitet nur mit dem abstrakten Typ `Shape`.

12.5. Abstrakte Klassen und Methoden

Eine **abstrakte Klasse** kann nicht instanziiert werden. Sie dient als Vorlage für abgeleitete Klassen und kann sowohl implementierte als auch abstrakte Member enthalten.

```
public abstract class Mitarbeiter
{
    private string _name;
    private DateTime _eintrittsdatum;

    protected Mitarbeiter(string name, DateTime eintrittsdatum)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));

        _name = name;
        _eintrittsdatum = eintrittsdatum;
    }

    public string Name => _name;
    public DateTime Eintrittsdatum => _eintrittsdatum;

    // Konkrete Methode - für alle Mitarbeiter gleich
    public int BerechneDienstjahre(DateTime stichtag)
    {
        return stichtag.Year - _eintrittsdatum.Year;
    }

    // Abstrakte Methode - muss in abgeleiteten Klassen implementiert werden
    public abstract decimal BerechneGehalt();

    public override string ToString()
    {
        return $"{_name} (seit {_eintrittsdatum:yyyy})";
    }
}

public class Festangestellter : Mitarbeiter
{
    private decimal _monatsgehalt;

    public Festangestellter(string name, DateTime eintrittsdatum, decimal monatsgehalt)
        : base(name, eintrittsdatum)
    {
    }
}
```

```

{
    if (monatsgehalt < 0)
        throw new ArgumentException("Gehalt darf nicht negativ sein", nameof(monatsgehalt));
    _monatsgehalt = monatsgehalt;
}

public override decimal BerechneGehalt()
{
    return _monatsgehalt;
}
}

public class Freelancer : Mitarbeiter
{
    private decimal _stundensatz;
    private int _gearbeiteteStunden;

    public Freelancer(string name, DateTime eintrittsdatum, decimal stundensatz)
        : base(name, eintrittsdatum)
    {
        if (stundensatz < 0)
            throw new ArgumentException("Stundensatz darf nicht negativ sein", nameof(
stundensatz));
        _stundensatz = stundensatz;
        _gearbeiteteStunden = 0;
    }

    public void StundenEintragen(int stunden)
    {
        if (stunden < 0)
            throw new ArgumentException("Stunden dürfen nicht negativ sein", nameof(stunden));
        _gearbeiteteStunden += stunden;
    }

    public override decimal BerechneGehalt()
    {
        return _stundensatz * _gearbeiteteStunden;
    }
}

```

Eigenschaften abstrakter Klassen:

- Können nicht mit `new` instanziiert werden
- Können sowohl abstrakte als auch konkrete Member enthalten
- Dürfen Konstruktoren haben (werden von abgeleiteten Klassen aufgerufen)
- Abgeleitete Klassen müssen alle abstrakten Member implementieren

12.6. Vererbung vs. Komposition

Ein häufiger Fehler ist die übermäßige Verwendung von Vererbung. Nicht jede "Ist-ein"-Beziehung ist auch eine Vererbungsbeziehung. Die Faustregel lautet:

- **Vererbung:** "Ist-ein" (is-a) Beziehung mit echter Spezialisierung
- **Komposition:** "Hat-ein" (has-a) Beziehung

```
// FALSCH: Vererbung für Eigenschaften missbraucht
public class Motor
{
    public int Ps { get; set; }
}

public class Auto : Motor // Ein Auto IST KEIN Motor!
{
    public string Marke { get; set; }
}

// RICHTIG: Komposition
public class Motor
{
    public int Ps { get; }
    public string Typ { get; }

    public Motor(string typ, int ps)
    {
        Typ = typ;
        Ps = ps;
    }
}

public class Auto
{
    public string Marke { get; }
    public Motor Motor { get; } // Auto HAT einen Motor

    public Auto(string marke, Motor motor)
    {
        Marke = marke;
        Motor = motor ?? throw new ArgumentNullException(nameof(motor));
    }
}
```

Problematische Vererbungshierarchien vermeiden:

```
// PROBLEMATISCH: Zu tiefe Hierarchien
public class Tier { }
public class Saeugetier : Tier { }
public class Hund : Saeugetier { }
public class Schaeferhund : Hund { }
public class DeutscherSchaeferhund : Schaeferhund { } // Zu tief!

// BESSER: Flache Hierarchie mit Komposition
public class Tier
{
    public string Name { get; }
    public Tierart Art { get; } // Komposition statt tiefer Vererbung

    public Tier(string name, Tierart art)
    {
        Name = name;
        Art = art;
    }
}
```

```
public enum Tierart { Hund, Katze, Vogel, /* ... */ }
```

Wann Vererbung sinnvoll ist:

- Die abgeleitete Klasse ist eine **echte Spezialisierung** der Basisklasse
- Es gibt gemeinsames Verhalten, das in der Basisklasse implementiert werden kann
- Polymorphie wird benötigt (verschiedene Typen einheitlich behandeln)
- Die "Ist-ein"-Beziehung gilt in **beide Richtungen** (jeder Student ist eine Person)

Wann Komposition besser ist:

- Die Beziehung ist eher "Hat-ein" als "Ist-ein"
- Sie möchten Implementierungsdetails austauschbar halten
- Die Hierarchie würde zu tief werden
- Mehrere Vererbungspfade wären nötig (C# unterstützt keine Mehrfachvererbung bei Klassen)

12.7. Die `sealed`-Klasse und `sealed`-Methoden

Mit `sealed` kann verhindert werden, dass von einer Klasse weitergeerbt wird oder dass eine Methode weiter überschrieben wird:

```
public sealed class KonkreteImplementierung
{
    // Von dieser Klasse kann nicht mehr geerbt werden
}

public class Basisklasse
{
    protected virtual void MacheEtwas() { }
}

public class Mittelklasse : Basisklasse
{
    protected sealed override void MacheEtwas()
    {
        // Diese Methode kann in weiteren abgeleiteten Klassen
        // nicht mehr überschrieben werden
    }
}

// FEHLER: public class Unterklasse : KonkreteImplementierung { }
```

12.8. Zusammenfassung Vererbung

- **Vererbung** ermöglicht Code-Wiederverwendung und Polymorphie
- `virtual/override` für überschreibbare Methoden
- `protected` für Member, die in abgeleiteten Klassen sichtbar sein sollen
- **Abstrakte Klassen** definieren gemeinsame Basisfunktionalität und erzwingen Implementierung in abgeleiteten Klassen
- **Polymorphie** ermöglicht einheitliche Behandlung verschiedener Typen

- **Komposition bevorzugen** wenn keine echte "Ist-ein"-Beziehung vorliegt
- Flache Hierarchien sind besser als tiefe

Kapitel 13. Interfaces

Ein Interface definiert einen *Vertrag* - eine Menge von Methoden und Properties, die eine Klasse implementieren muss. Interfaces enthalten keine Implementierung, sondern legen nur fest, *was* eine Klasse können muss, nicht *wie* es umgesetzt wird.

13.1. Wann braucht man Interfaces?

Interfaces sind nicht immer notwendig. Sie werden dann sinnvoll, wenn:

1. **Polymorphie über Klassengrenzen hinweg:** Verschiedene Klassen, die keine gemeinsame Basisklasse haben, sollen einheitlich behandelt werden.
2. **Unabhängigkeit von Implementierungen:** Der Aufrufer soll nicht von einer konkreten Klasse abhängen, sondern nur von den benötigten Fähigkeiten.
3. **Testbarkeit:** In Unit-Tests können leicht Mock-Objekte erstellt werden, die das Interface implementieren.
4. **Mehrere Verträge:** Eine Klasse kann mehrere Interfaces implementieren (im Gegensatz zur Einfachvererbung bei Klassen).

13.2. Praktisches Beispiel: `IUsable` mit Waffen und Heiltränken

Betrachten wir ein Rollenspiel. Ein Spieler kann verschiedene Gegenstände verwenden: Waffen zum Angreifen und Heiltränke zur Heilung. Diese Gegenstände haben keine gemeinsame Basisklasse - sie sind völlig unterschiedliche Konzepte. Aber sie haben eines gemeinsam: Sie können *verwendet* werden.

Das Interface `IUsable` definiert diesen Vertrag:

```
public interface IUsable
{
    string Name { get; }
    void Use(Player target);
    bool CanUse(Player target);
}
```

Nun implementieren verschiedene Gegenstände dieses Interface:

```
public class Weapon : IUsable
{
    private string _name;
    private int _damage;

    public Weapon(string name, int damage)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));
        if (damage <= 0)
            throw new ArgumentOutOfRangeException(nameof(damage), "Schaden muss positiv sein");

        _name = name;
        _damage = damage;
    }

    public string Name => _name;
```

```
public void Use(Player target)
{
    if (!CanUse(target))
        throw new InvalidOperationException($"{_name} kann nicht auf {target.Name} angewendet
werden");

    target.TakeDamage(_damage);
    Console.WriteLine($"{_name} verursacht {_damage} Schaden bei {target.Name}");
}

public bool CanUse(Player target)
{
    return target != null && target.IsAlive;
}
}

public class HealingPotion : IUsable
{
    private string _name;
    private int _healAmount;
    private int _charges;

    public HealingPotion(string name, int healAmount, int charges)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Name darf nicht leer sein", nameof(name));
        if (healAmount <= 0)
            throw new ArgumentOutOfRangeException(nameof(healAmount), "Heilung muss positiv sein");
        if (charges <= 0)
            throw new ArgumentOutOfRangeException(nameof(charges), "Anzahl muss positiv sein");

        _name = name;
        _healAmount = healAmount;
        _charges = charges;
    }

    public string Name => _name;

    public void Use(Player target)
    {
        if (!CanUse(target))
            throw new InvalidOperationException($"{_name} kann nicht verwendet werden");

        target.Heal(_healAmount);
        _charges--;
        Console.WriteLine($"{_name} heilt {target.Name} um {_healAmount} HP ({_charges} Anwendungen
verbleibend)");
    }

    public bool CanUse(Player target)
    {
        return target != null && _charges > 0 && target.Health < target.MaxHealth;
    }
}

public class Player
{

```



```
private string _name;
private int _health;
private int _maxHealth;

public Player(string name, int maxHealth)
{
    if (string.IsNullOrEmpty(name))
        throw new ArgumentException("Name darf nicht leer sein", nameof(name));
    if (maxHealth <= 0)
        throw new ArgumentOutOfRangeException(nameof(maxHealth), "MaxHealth muss positiv sein");

    _name = name;
    _maxHealth = maxHealth;
    _health = maxHealth;
}

public string Name => _name;
public int Health => _health;
public int MaxHealth => _maxHealth;
public bool IsAlive => _health > 0;

public void TakeDamage(int damage)
{
    if (damage < 0) throw new ArgumentOutOfRangeException(nameof(damage));
    _health = Math.Max(0, _health - damage);
}

public void Heal(int amount)
{
    if (amount < 0) throw new ArgumentOutOfRangeException(nameof(amount));
    _health = Math.Min(_maxHealth, _health + amount);
}
}
```

13.3. Die List mit Interface-Typisierung

Der entscheidende Vorteil zeigt sich, wenn wir eine Liste von verwendbaren Gegenständen verwalten:

```
public class Inventory
{
    private List<IUsable> _items = new List<IUsable>();

    public void AddItem(IUsable item)
    {
        if (item == null)
            throw new ArgumentNullException(nameof(item));

        _items.Add(item);
    }

    public void UseItem(int index, Player target)
    {
        if (index < 0 || index >= _items.Count)
            throw new ArgumentOutOfRangeException(nameof(index));

        IUsable item = _items[index];
    }
}
```

```

        if (item.CanUse(target))
        {
            item.Use(target);
        }
        else
        {
            Console.WriteLine($"{item.Name} kann nicht verwendet werden.");
        }
    }

    public void ListItems()
    {
        Console.WriteLine("Inventar:");
        for (int i = 0; i < _items.Count; i++)
        {
            Console.WriteLine($" [{i}] {_items[i].Name}");
        }
    }
}

```

Anwendung:

```

var spieler = new Player("Held", 100);
var inventar = new Inventory();

// Verschiedene Gegenstände hinzufügen - alle implementieren IUsable
inventar.AddItem(new Weapon("Schwert", 25));
inventar.AddItem(new HealingPotion("Kleiner Heiltrank", 30, 3));
inventar.AddItem(new Weapon("Bogen", 15));
inventar.AddItem(new HealingPotion("Großer Heiltrank", 50, 1));

inventar.ListItems();

// Gegenstände verwenden - der Aufrufer muss nicht wissen, ob es eine Waffe oder ein Trank ist
inventar.UseItem(1, spieler); // Heiltrank verwenden
inventar.UseItem(0, spieler); // Schwert verwenden (auf sich selbst - Demospiel!)

```

Ohne Interface müssten wir separate Listen für Waffen und Heiltränke führen oder eine gemeinsame Basisklasse erzwingen, die semantisch keinen Sinn ergibt (eine Waffe ist kein Heiltrank).

13.4. Interfaces vs. Abstrakte Klassen

Sowohl Interfaces als auch abstrakte Klassen ermöglichen Polymorphie. Wann verwendet man was?

Kriterium	Interface	Abstrakte Klasse
Implementierung	Keine (nur Signaturen)	Kann teilweise Implementierung enthalten
Vererbung	Mehrere Interfaces möglich	Nur eine Basisklasse
Zustand	Keine Felder (nur Properties)	Kann Felder enthalten
Konstruktoren	Nicht möglich	Möglich

Kriterium	Interface	Abstrakte Klasse
Versionierung	Änderungen brechen alle Implementierungen	Neue Methoden können Standardimplementierung haben
"Ist-ein" Beziehung	"Kann etwas tun"	"Ist eine Art von"

Faustregel:

- **Interface:** Wenn verschiedene Klassen eine gemeinsame *Fähigkeit* haben (z.B. `IUsable`, `IComparable`, `IDisposable`)
- **Abstrakte Klasse:** Wenn Klassen eine gemeinsame *Identität* und gemeinsame Basisfunktionalität haben (z.B. `Mitarbeiter` mit `BerechneGehalt()`)

Beispiel für abstrakte Klasse:

```
public abstract class Mitarbeiter
{
    protected string _name;
    protected decimal _basisgehalt;

    protected Mitarbeiter(string name, decimal basisgehalt)
    {
        _name = name;
        _basisgehalt = basisgehalt;
    }

    public string Name => _name;

    // Konkrete Methode - für alle Mitarbeiter gleich
    public string ErstelleBericht()
    {
        return $"Bericht von {_name}";
    }

    // Abstrakte Methode - muss von abgeleiteten Klassen implementiert werden
    public abstract decimal BerechneGehalt();
}

public class Festangestellter : Mitarbeiter
{
    public Festangestellter(string name, decimal basisgehalt)
        : base(name, basisgehalt) { }

    public override decimal BerechneGehalt() => _basisgehalt;
}

public class Freelancer : Mitarbeiter
{
    private int _stunden;
    private decimal _stundensatz;

    public Freelancer(string name, int stunden, decimal stundensatz)
        : base(name, 0)
    {
        _stunden = stunden;
    }
}
```

```

        _stundensatz = stundensatz;
    }

    public override decimal BerechneGehalt() => _stunden * _stundensatz;
}

```

13.5. Funktionale Interfaces und Delegates

Ein **Delegate** ist ein Typsicherer Funktionszeiger - er definiert, welche Signatur eine Methode haben muss, um dem Delegate zu entsprechen. In diesem Sinne sind Delegates mit Interfaces verwandt: Beide definieren einen Vertrag.

Einfaches Delegate-Beispiel:

```

// Delegate-Deklaration
public delegate int BerechneDelegate(int a, int b);

public class Rechner
{
    public int Addiere(int a, int b) => a + b;
    public int Multipliziere(int a, int b) => a * b;
}

var rechner = new Rechner();
BerechneDelegate operation = rechner.Addiere;
int ergebnis = operation(5, 3); // 8

```

13.5.1. Vordefinierte funktionale Interfaces

Table 1. NET bietet vordefinierte Delegate-Typen, die als "funktionale Interfaces" fungieren:

Delegate-Typ	Signatur	Verwendung
Action	void()	Methode ohne Parameter und Rückgabewert
Action<T>	void(T obj)	Methode mit einem Parameter
Action<T1, T2>	void(T1, T2)	Methode mit zwei Parametern
Func<TResult>	TResult()	Methode ohne Parameter mit Rückgabewert
Func<T, TResult>	TResult(T arg)	Methode mit einem Parameter und Rückgabewert
Func<T1, T2, TResult>	TResult(T1, T2)	Methode mit zwei Parametern und Rückgabewert
Predicate<T>	bool(T obj)	Methode, die true/false zurückgibt

Beispiele mit Func und Action:

```

// Func für Berechnungen
Func<int, int, int> addiere = (a, b) => a + b;
Func<int, int, int> multipliziere = (a, b) => a * b;

int summe = addiere(5, 3); // 8
int produkt = multipliziere(5, 3); // 15

// Func mit LINQ
List<int> zahlen = new List<int> { 1, 2, 3, 4, 5 };

```

```
Func<int, bool> istGerade = n => n % 2 == 0;
var geradeZahlen = zahlen.Where(istGerade); // 2, 4

// Action für Seiteneffekte
Action<string> begruesse = name => Console.WriteLine($"Hallo, {name}!");
begruesse("Welt");

// Action als Callback
Action<string> logInfo = msg => Console.WriteLine($"[INFO] {DateTime.Now}: {msg}");
Action<string> logError = msg => Console.WriteLine($"[ERROR] {DateTime.Now}: {msg}");

void FuehreAus(string daten, Action<string> logger)
{
    logger($"Verarbeite: {daten}");
}

FuehreAus("Testdaten", logInfo);
```

13.5.2. Predicate<T> VS. Func<T, bool>

Das `Predicate<T>`-Delegate ist ein spezieller Fall von `Func<T, bool>`:

```
// Beide sind funktional äquivalent:
Predicate<int> predicate = n => n > 0;
Func<int, bool> func = n => n > 0;
```

Wichtiger Hinweis: In der modernen C#-Entwicklung wird `Predicate<T>` zunehmend seltener verwendet. Die meisten .NET-APIs, insbesondere LINQ, bevorzugen `Func<T, bool>`:

```
// LINQ verwendet Func<T, bool>
List<int> zahlen = new List<int> { -2, -1, 0, 1, 2, 3 };

// Mit Func (empfohlen, Standard in LINQ)
Func<int, bool> istPositiv = n => n > 0;
var positiveZahlen = zahlen.Where(istPositiv).ToList();

// Predicate funktioniert nicht direkt mit LINQ-Where
Predicate<int> predicateIstPositiv = n => n > 0;
// zahlen.Where(predicateIstPositiv); // Kompilierfehler!
// Muss umgewandelt werden:
var positiveZahlen2 = zahlen.Where(n => predicateIstPositiv(n)).ToList();

// List<T>.FindAll() akzeptiert Predicate (historisch bedingt)
var positiveMitPredicate = zahlen.FindAll(predicateIstPositiv);
```

Empfehlung: Verwenden Sie `Func<T, bool>` anstelle von `Predicate<T>` für Konsistenz mit LINQ und modernen .NET-APIs. `Predicate<T>` existiert hauptsächlich aus historischen Gründen (eingeführt vor LINQ).

13.6. Repository-Pattern mit Interfaces

Ein klassischer Anwendungsfall für Interfaces ist das Repository-Pattern für Datenbankzugriffe:

```
public interface IRepository<T>
{
```

```
T GetById(int id);
void Add(T entity);
void Update(T entity);
void Delete(int id);
IEnumerable<T> GetAll();
}

public class StudentRepository : IRepository<Student>
{
    private readonly List<Student> _studenten = new List<Student>();

    public void Add(Student student)
    {
        if (student == null)
            throw new ArgumentNullException(nameof(student));

        _studenten.Add(student);
    }

    public void Delete(int id)
    {
        var student = GetById(id);
        if (student != null)
            _studenten.Remove(student);
    }

    public Student GetById(int id)
    {
        return _studenten.FirstOrDefault(s => s.Id == id);
    }

    public void Update(Student student)
    {
        // Implementierung...
    }

    public IEnumerable<Student> GetAll()
    {
        return _studenten.AsReadOnly();
    }
}
```

Der Vorteil: Der Service, der das Repository verwendet, hängt nur vom Interface ab:

```
public class StudentService
{
    private readonly IRepository<Student> _repository;

    public StudentService(IRepository<Student> repository)
    {
        _repository = repository ?? throw new ArgumentNullException(nameof(repository));
    }

    public Student FindeStudent(int id)
    {
        return _repository.GetById(id);
    }
}
```

Im Unit-Test kann nun ein Mock-Repository injiziert werden:

```
public class MockStudentRepository : IRepository<Student>
{
    private readonly List<Student> _daten = new List<Student>
    {
        new Student { Id = 1, Name = "Test Student" }
    };

    public Student GetById(int id) => _daten.FirstOrDefault(s => s.Id == id);
    public void Add(Student entity) => _daten.Add(entity);
    public void Update(Student entity) { /* Mock */ }
    public void Delete(int id) => _daten.RemoveAll(s => s.Id == id);
    public IEnumerable<Student> GetAll() => _daten;
}

// Im Test
var mockRepo = new MockStudentRepository();
var service = new StudentService(mockRepo);
// Jetzt kann getestet werden, ohne eine echte Datenbank zu benötigen
```

13.7. Zusammenfassung

- **Interfaces** definieren Verträge (Signaturen) ohne Implementierung
- Verwenden Sie Interfaces für **Polymorphie über Klassengrenzen hinweg** - wenn verschiedene Klassen eine gemeinsame Fähigkeit haben
- Eine Klasse kann **mehrere Interfaces** implementieren (im Gegensatz zur Einfachvererbung bei Klassen)
- **Abstrakte Klassen** (siehe Kapitel "Vererbung") eignen sich für echte "Ist-ein"-Beziehungen mit gemeinsamer Identität und Basisfunktionalität
- **Interfaces vs. Abstrakte Klassen:**
 - Interface: "Kann etwas tun" (Fähigkeit, z.B. `IUsable`, `IComparable`)
 - Abstrakte Klasse: "Ist eine Art von" (Identität, z.B. `Mitarbeiter`)
- **Delegates** (`Func`, `Action`) sind funktionale Interfaces für Callbacks und LINQ
- Bevorzugen Sie `Func<T, bool>` gegenüber `Predicate<T>` für Konsistenz mit LINQ

Kapitel 14. Datenbanken mit Entity Framework Core (EF Core)

Entity Framework Core (EF Core) ist ein moderner Object-Relational Mapper (ORM) für .NET. Er ermöglicht es, Datenbanken mithilfe von C#-Objekten zu verwalten und Abfragen direkt in C# (via LINQ) zu schreiben.

Wir verwenden in diesem Skriptum **SQLite**, eine leichtgewichtige, dateibasierte Datenbank, die keine Serverinstallation erfordert.

14.1. EF Core Pakete installieren

Um EF Core mit SQLite zu nutzen, müssen entsprechende NuGet-Pakete installiert werden:

```
# Core Framework und SQLite Support
dotnet add package Microsoft.EntityFrameworkCore.Sqlite

# Werkzeuge für Migrations (CLI Tools)
dotnet add package Microsoft.EntityFrameworkCore.Design
```

14.2. Das Domänenmodell und der DbContext

Das Herzstück von EF Core ist der `DbContext`. Er repräsentiert eine Sitzung mit der Datenbank und ermöglicht das Abfragen und Speichern von Daten.

```
using Microsoft.EntityFrameworkCore;

// Domänenmodell
public class Student
{
    public int Id { get; set; } // EF Core erkennt "Id" automatisch als Primärschlüssel
    public string Name { get; set; } = string.Empty;
    public int Geburtsjahr { get; set; }
}

// Datenbankkontext
public class SchulContext : DbContext
{
    public DbSet<Student> Studenten { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        // Verbindung zur SQLite-Datei herstellen
        optionsBuilder.UseSqlite("Data Source=schule.db");
    }
}
```

14.3. Schema-Migrations

Migrations ermöglichen es, das Datenbank-Schema synchron mit den C#-Klassen zu halten. Jede Änderung am Modell (z.B. ein neues Feld) wird in eine Migrations-Datei übersetzt.

14.3.1. Die erste Migration erstellen

Nachdem der `DbContext` definiert wurde, muss die Datenbank initialisiert werden:

```
# Erstellt den Code für das Tabellenschema
dotnet ef migrations add InitialCreate
```



```
# Wendet die Migration auf die Datenbank an (erzeugt schule.db)
dotnet ef database update
```

Hinweis: Sollte der Befehl `dotnet ef` nicht gefunden werden, muss das globale Tool installiert werden: `dotnet tool install --global dotnet-ef`.

14.3.2. Schema-Änderungen durchführen

Wenn Sie eine Klasse ändern (z.B. ein Feld `Email` zu `Student` hinzufügen), wiederholen Sie den Vorgang: 1. `dotnet ef migrations add AddStudentEmail` 2. `dotnet ef database update`

14.4. CRUD-Operationen

Dank EF Core können Datenbankoperationen wie normale Listenoperationen durchgeführt werden:

```
using var context = new SchulContext();

// 1. Create (Hinzufügen)
var neuerStudent = new Student { Name = "Max Mustermann", Geburtsjahr = 2005 };
context.Studenten.Add(neuerStudent);
context.SaveChanges(); // WICHTIG: Erst hier wird der SQL-Befehl ausgeführt!

// 2. Read (Abfragen)
var student = context.Studenten
    .FirstOrDefault(s => s.Name == "Max Mustermann");

// 3. Update (Aktualisieren)
if (student != null)
{
    student.Geburtsjahr = 2006;
    context.SaveChanges();
}

// 4. Delete (Löschen)
if (student != null)
{
    context.Studenten.Remove(student);
    context.SaveChanges();
}
```

14.5. 1:n Beziehungen (One-to-Many)

In einer 1:n Beziehung kann ein Datensatz der Tabelle A mit mehreren Datensätzen der Tabelle B verknüpft sein, aber ein Datensatz der Tabelle B gehört zu genau einem Datensatz der Tabelle A. Ein klassisches Beispiel ist die Beziehung zwischen einer Abteilung und ihren Mitarbeitern.

```
public class Abteilung
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    // Navigation Property: Eine Abteilung hat viele Mitarbeiter
    public List<Mitarbeiter> Mitarbeiter { get; set; } = new();
}
```

```

}

public class Mitarbeiter
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    // Fremdschlüssel (Foreign Key)
    public int AbteilungId { get; set; }

    // Navigation Property: Ein Mitarbeiter gehört zu einer Abteilung
    public Abteilung Abteilung { get; set; } = null!;
}

```

EF Core erkennt diese Beziehung automatisch anhand der Navigation Properties und des Fremdschlüssels (`AbteilungId`).

14.6. n:m Beziehungen (Many-to-Many)

In einer n:m Beziehung können mehrere Datensätze der Tabelle A mit mehreren Datensätzen der Tabelle B verknüpft sein. Ein Beispiel hierfür ist die Beziehung zwischen Studenten und Kursen: Ein Student kann viele Kurse besuchen, und ein Kurs kann von vielen Studenten besucht werden.

```

public class Student
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    // Ein Student besucht viele Kurse
    public List<Kurs> Kurse { get; set; } = new();
}

public class Kurs
{
    public int Id { get; set; }
    public string Titel { get; set; } = string.Empty;

    // Ein Kurs hat viele Studenten
    public List<Student> Studenten { get; set; } = new();
}

```

Seit EF Core 5.0 erstellt das Framework automatisch eine Zwischentabelle (Join-Table) in der Datenbank, ohne dass diese explizit als Klasse im C#-Code definiert werden muss.

14.7. Laden von Beziehungen (Eager Loading)

Standardmäßig lädt EF Core keine verwandten Daten (Lazy Loading ist standardmäßig deaktiviert). Um Beziehungen mitzuladen, verwenden wir die `Include`-Methode. Dies wird als **Eager Loading** bezeichnet.

```

using Microsoft.EntityFrameworkCore;

// Mitarbeiter inklusive ihrer Abteilung laden
var mitarbeiterListe = context.Mitarbeiter
    .Include(m => m.Abtteilung)

```

```
.ToList();

// Abfrage mit Filterung über die Beziehung
var itMitarbeiter = context.Mitarbeiter
    .Include(m => m.Abteilung)
    .Where(m => m.Abteilung.Name == "IT")
    .ToList();
```

14.8. Best Practices für EF Core

1. **Unit of Work:** Verwenden Sie `SaveChanges()` gesammelt am Ende einer logischen Operation, nicht nach jeder einzelnen Zeile.
2. **AsNoTracking:** Verwenden Sie `.AsNoTracking()`, wenn Sie Daten nur lesen möchten (erhöht die Performance).
3. **Fail-Fast:** Validieren Sie Daten bereits im Domänenmodell, bevor Sie `SaveChanges()` aufrufen.
4. **Migrations einchecken:** Die Migrations-Dateien im Verzeichnis `Migrations/` gehören in das Git-Repository, da sie die Historie des Schemas dokumentieren.

Kapitel 15. LINQ (Language Integrated Query)

LINQ ist ein mächtiges Feature von C#, das es ermöglicht, Daten aus verschiedenen Quellen (Collections, Datenbanken, XML, etc.) auf einheitliche Weise zu abzufragen und zu manipulieren.

15.1. Grundlegende LINQ-Operationen

```
using System.Linq;

List<int> zahlen = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// Filterung (Where)
var geradeZahlen = zahlen.Where(z => z % 2 == 0);

// Projektion (Select)
var quadrate = zahlen.Select(z => z * z);

// Sortierung (OrderBy, OrderByDescending)
var absteigend = zahlen.OrderByDescending(z => z);

// Aggregationen
int summe = zahlen.Sum();
double durchschnitt = zahlen.Average();
int maximum = zahlen.Max();
int minimum = zahlen.Min();
int anzahl = zahlen.Count();

// Existenzprüfungen
bool hatGerade = zahlen.Any(z => z % 2 == 0);
bool allePositiv = zahlen.All(z => z > 0);

// Erstes Element
int ersteGerade = zahlen.First(z => z % 2 == 0);
int ersteOrderDefault = zahlen.FirstOrDefault(z => z > 100); // 0 wenn nicht gefunden

// Selektion (Findet ein einzelnes Element)
int dieFuenf = zahlen.Single(z => z == 5);
```

15.2. LINQ with Objects

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Geburtsjahr { get; set; }
    public string Studiengang { get; set; }
}

// Beispielabfragen
List<Student> studenten = LadeStudenten();

// Alle Studenten eines Studiengangs
var informatikStudenten = studenten
    .Where(s => s.Studiengang == "Informatik")
    .OrderBy(s => s.Name);

// Namen aller Studenten
```

```
var namen = studenten.Select(s => s.Name);

// Gruppierung nach Studiengang
var gruppiert = studenten
    .GroupBy(s => s.Studiengang)
    .Select(g => new
    {
        Studiengang = g.Key,
        Anzahl = g.Count(),
        DurchschnittsAlter = DateTime.Now.Year - g.Average(s => s.Geburtsjahr)
    });

// Komplexe Abfrage mit mehreren Operationen
var ergebnis = studenten
    .Where(s => s.Geburtsjahr > 1990)
    .OrderBy(s => s.Name)
    .Take(10)
    .Select(s => $"{s.Name} ({DateTime.Now.Year - s.Geburtsjahr} Jahre)")
    .ToList();
```

15.3. Query-Syntax vs. Method-Syntax

LINQ bietet zwei Syntax-Varianten:

```
// Method-Syntax (Lambda-Syntax)
var ergebnis1 = studenten
    .Where(s => s.Geburtsjahr > 1990)
    .OrderBy(s => s.Name)
    .Select(s => s.Name);

// Query-Syntax (SQL-ähnlich)
var ergebnis2 = from s in studenten
                 where s.Geburtsjahr > 1990
                 orderby s.Name
                 select s.Name;

// Beide Varianten sind äquivalent!
```

Kapitel 16. Exception Handling

16.1. Grundlagen

Exceptions (Ausnahmen) sind Fehler, die zur Laufzeit auftreten. In C# werden Exceptions mit `try-catch-finally` behandelt:

```
public void BeispielMethode()
{
    try
    {
        // Code, der eine Exception werfen könnte
        int ergebnis = 10 / 0;
    }
    catch (DivideByZeroException ex)
    {
        // Spezifische Behandlung für Division durch Null
        Console.WriteLine($"Fehler: {ex.Message}");
    }
    catch (Exception ex)
    {
        // Allgemeine Behandlung
        Console.WriteLine($"Unbekannter Fehler: {ex.Message}");
    }
    finally
    {
        // Wird immer ausgeführt, auch wenn eine Exception auftrat
        // Gut für Cleanup-Operationen
    }
}
```

16.2. Eigene Exceptions

```
public class ValidationException : Exception
{
    public ValidationException(string message) : base(message)
    {
    }

    public ValidationException(string message, Exception innerException)
        : base(message, innerException)
    {
    }
}

public class StudentService
{
    public void RegistriereStudent(Student student)
    {
        if (student == null)
            throw new ArgumentNullException(nameof(student));

        if (string.IsNullOrEmpty(student.Name))
            throw new ValidationException("Studentenname ist erforderlich");

        // Registrierung...
    }
}
```

Kapitel 17. Razor

Razor ist eine Syntax für die Erstellung dynamischer Webseiten mit C#. Sie ermöglicht die nahtlose Einbettung von C#-Code in HTML.

17.1. Razor-Grundlagen

Eine Razor-Datei hat die Endung `.cshtml`. Sie enthält HTML mit eingebettetem C#-Code:

```
@page
@model IndexModel

<!DOCTYPE html>
<html>
<head>
    <title>@Model.Seitentitel</title>
</head>
<body>
    <h1>Willkommen, @Model.Benutzername!</h1>

    @if (Model.IstAngemeldet)
    {
        <p>Sie sind angemeldet.</p>
    }
    else
    {
        <p>Bitte melden Sie sich an.</p>
    }

    <ul>
    @foreach (var eintrag in Model.Eintraege)
    {
        <li>@eintrag.Name - @eintrag.Datum.ToShortDateString()</li>
    }
    </ul>
</body>
</html>
```

17.2. Razor-Page Model

Das Page Model enthält die Logik für eine Razor-Page:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class IndexModel : PageModel
{
    private readonly IStudentService _studentService;

    public IndexModel(IStudentService studentService)
    {
        _studentService = studentService ?? throw new ArgumentNullException(nameof(
studentService));
    }

    public string Seitentitel { get; set; } = "Meine Seite";
    public string Benutzername { get; set; }
    public bool IstAngemeldet { get; set; }
```

```
public List<Eintrag> Eintraege { get; set; } = new List<Eintrag>();

public void OnGet()
{
    // Wird bei GET-Request aufgerufen
    Eintraege = _studentService.LadeAlleEintraege();
}

public IActionResult OnPost(int id)
{
    // Wird bei POST-Request aufgerufen
    try
    {
        _studentService.LoescheEintrag(id);
        return RedirectToPage("/Index");
    }
    catch (Exception ex)
    {
        ModelState.AddModelError("", $"Fehler beim Löschen: {ex.Message}");
        return Page();
    }
}
}
```

17.3. Razor-Komponenten

Razor-Komponenten (Blazor) ermöglichen die Erstellung wiederverwendbarer UI-Elemente:

```
<!-- StudentCard.razor -->
<div class="student-card">
    <h3>@Student.Name</h3>
    <p>Matrikelnummer: @Student.Matrikelnummer</p>
    <button @onclick="OnDetailsClick">Details</button>
</div>

@code {
    [Parameter]
    public Student Student { get; set; }

    [Parameter]
    public EventCallback<int> OnDetails { get; set; }

    private async Task OnDetailsClick()
    {
        await OnDetails.InvokeAsync(Student.Id);
    }
}
```


Kapitel 18. Razor mit HTMX

HTMX ist eine JavaScript-Bibliothek, die es ermöglicht, moderne interaktive Webanwendungen zu erstellen, ohne komplexes JavaScript zu schreiben. In Kombination mit Razor erlaubt es die Erstellung von Server-seitig gerenderten Webanwendungen mit dynamischem Verhalten.

18.1. Was ist HTMX?

HTMX erweitert HTML um Attribute, die AJAX-Requests, CSS-Transitions und WebSocket-Verbindungen direkt im Markup ermöglichen:

```
<!-- Standard HTML-Button -->
<button>Klick mich</button>

<!-- HTMX-erweiterter Button -->
<button hx-get="/api/studenten"
        hx-target="#studenten-liste"
        hx-trigger="click">
    Studenten laden
</button>
```

18.2. HTMX in Razor Pages

```
@page
@model StudentenModel

<h1>Studentenverwaltung</h1>

<!-- Teil der Seite, der dynamisch aktualisiert wird -->
<div id="studenten-container">
    <partial name="_StudentenListe" model="Model.Studenten" />
</div>

<!-- Formular mit HTMX für serverseitige Verarbeitung -->
<form hx-post="/Studenten"
      hx-target="#studenten-container"
      hx-swap="innerHTML">

    <div>
        <label for="name">Name:</label>
        <input type="text" id="name" name="Name" required />
    </div>

    <div>
        <label for="geburtsjahr">Geburtsjahr:</label>
        <input type="number" id="geburtsjahr" name="Geburtsjahr" required />
    </div>

    <button type="submit">Student hinzufügen</button>
</form>

<!-- Löschen mit Bestätigung -->
@foreach (var student in Model.Studenten)
{
    <div class="student-eintrag">
        <span>@student.Name</span>
        <button hx-delete="/Studenten?id=@student.Id"
                hx-target="#studenten-container">
```

```

        hx-confirm="Möchten Sie @student.Name wirklich löschen?"
        hx-swap="outerHTML">
        Löschen
    </button>
</div>
}

```

18.3. PageModel für HTMX

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class StudentenModel : PageModel
{
    private readonly IStudentRepository _repository;

    public StudentenModel(IStudentRepository repository)
    {
        _repository = repository ?? throw new ArgumentNullException(nameof(repository));
    }

    public List<Student> Studenten { get; set; } = new List<Student>();

    public void OnGet()
    {
        Studenten = _repository.GetAll();
    }

    public IActionResult OnPost(StudentInput input)
    {
        if (!ModelState.IsValid)
        {
            return Partial("_StudentenListe", _repository.GetAll());
        }

        try
        {
            var student = new Student(input.Name, input.Geburtsjahr);
            _repository.Add(student);

            // Nur den aktualisierten Teil zurückgeben
            return Partial("_StudentenListe", _repository.GetAll());
        }
        catch (Exception ex)
        {
            ModelState.AddModelError("", ex.Message);
            return Partial("_StudentenListe", _repository.GetAll());
        }
    }

    public IActionResult OnDelete(int id)
    {
        try
        {
            _repository.Delete(id);
            return Partial("_StudentenListe", _repository.GetAll());
        }
        catch (Exception ex)
        {

```

```

    {
        // HTMX wird den Fehler anzeigen
        return StatusCode(500, ex.Message);
    }
}

public class StudentInput
{
    public string Name { get; set; }
    public int Geburtsjahr { get; set; }
}

```

18.4. HTMX-Partial Views

```

<!-- _StudentenListe.cshtml -->
@model List<Student>

<div id="studenten-container">
    @if (Model.Any())
    {
        <table class="table">
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Geburtsjahr</th>
                    <th>Aktionen</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var student in Model)
                {
                    <tr>
                        <td>@student.Name</td>
                        <td>@student.Geburtsjahr</td>
                        <td>
                            <button hx-delete="/Studenten?id=@student.Id"
                                hx-target="#studenten-container"
                                hx-confirm="Wirklich löschen?"
                                class="btn btn-danger">
                                    Löschen
                                </button>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    }
    else
    {
        <p>Keine Studenten vorhanden.</p>
    }
</div>

```

18.5. HTMX-Trigger und Events

HTMX bietet vielfältige Trigger-Optionen:

```

<!-- Trigger beim Laden der Seite -->
<div hx-get="/api/status"
      hx-trigger="load"
      hx-target="this">
  Lade Status...
</div>

<!-- Trigger bei Eingabe (debounced) -->
<input type="search"
       name="suchbegriff"
       hx-get="/api/suche"
       hx-trigger="keyup changed delay:500ms"
       hx-target="#suchergebnisse"
       placeholder="Suchen..." />

<!-- Trigger in Intervallen -->
<div hx-get="/api/aktuelle-uhrzeit"
      hx-trigger="every 5s"
      hx-target="this">
  @DateTime.Now.ToString("HH:mm:ss")
</div>

<!-- Trigger bei Sichtbarwerden -->
<div hx-get="/api/mehr-daten"
      hx-trigger="revealed"
      hx-target="this">
  Mehr laden...
</div>

```

18.6. HTMX mit Animationen

```

<!-- CSS für Transition -->
<style>
.student-eintrag {
  transition: all 0.3s ease;
  opacity: 1;
}

.student-eintrag.htmx-swapping {
  opacity: 0;
  transform: translateX(20px);
}

.student-eintrag.htmx-added {
  opacity: 0;
  transform: translateX(-20px);
}
</style>

<!-- HTML mit HTMX -->
<div id="studenten-liste" hx-swap="innerHTML swap:0.3s">
  <!-- Inhalt wird hier dynamisch geladen -->
</div>

<!-- Formular mit Feedback -->
<form hx-post="/Studenten"
      hx-target="#studenten-liste"
      hx-swap="beforeend">

```

```
hx-on::after-request="this.reset()">>

<input type="text" name="Name" required />
<button type="submit">Hinzufügen</button>
</form>
```

18.7. Vorteile von Razor + HTMX

1. **Server-seitiges Rendering:** Keine komplexe Client-Logik nötig
2. **Progressive Enhancement:** Funktioniert auch ohne JavaScript
3. **Einfachheit:** Weniger Code als bei SPAs (Single Page Applications)
4. **Performance:** Nur Teile der Seite werden neu geladen
5. **C#-Fokus:** Volle Nutzung der C#-Ökosystem-Stärken

18.8. Best Practices für Razor + HTMX

1. **Partial Views verwenden:** Erstellen Sie wiederverwendbare Partial Views für dynamische Teile
2. **Fail-Fast beibehalten:** Validieren Sie auch in HTMX-Requests sofort
3. **Rückgabecodes:** Verwenden Sie passende HTTP-Statuscodes (200, 400, 500)
4. **UX beachten:** Zeigen Sie Ladezustände und Fehlermeldungen an
5. **Testbarkeit:** Testen Sie die Endpunkte wie normale API-Controller

```
// Beispiel für gute HTMX-Integration mit Fail-Fast
public class HtmxStudentController : Controller
{
    private readonly IStudentRepository _repository;

    public HtmxStudentController(IStudentRepository repository)
    {
        _repository = repository ?? throw new ArgumentNullException(nameof(repository));
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Erstelle(StudentInput input)
    {
        // Fail-Fast: Validierung
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        try
        {
            var student = new Student(input.Name, input.Geburtsjahr);
            _repository.Add(student);

            // Nur die neue Zeile zurückgeben
            return PartialView("_StudentRow", student);
        }
        catch (ArgumentException ex)
        {
            return BadRequest(ex.Message);
        }
        catch (Exception ex)
        {
            // ...
        }
    }
}
```

```
{  
    // Logging...  
    return StatusCode(500, "Interner Serverfehler");  
}  
}
```